Engineering Math HW

Apurv Bhushan

April 2026

Q2) Numerical Derivative

Use the definition of derivative to calculate the derivative of sin(x) at x = 1.

a) Compare that to the exact value (exact value is cos(1)).

b) Plot error vs h (log-log plot) and note that there is a best h because of the competition between method error and roundoff error.

c) Challenge: Using the Matlab resolution of Matlab's smallest number "eps", estimate (by analysis) the best h for this calculation and compare it with the minimum on your graph.

Solution

a)

According to the definition of the derivative:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} \tag{1}$$

The value of the derivative of sin(x) at x = 1 comes out to be 0.5403 and the value of cos(1), which is the analytical derivative, also comes out to be 0.5403.

The difference is that Matlab calculates to a higher decimal place.

On calculating the absolute error:

$$\text{error} = |f'_{\text{definition}} - f'_{\text{analytical}}| \tag{2}$$

The value comes out to be 4.2075e-7, which is a significantly small number indicating the accuracy of the derivative definition.

b)

Graphical Output:



Figure 1: Log-log plot of error vs h for numerical derivative of sin(x) at x = 1

From the graph, the "best" h comes out to be 4.64e-9 with an error of 1.8e-9.

Q5) Make Up a Problem

Cook up, or find, some ODE, or set of first order ODEs, and solve them and explore something hopefully interesting about the solution.

Solution

For this problem, I chose the following differential equation:

$$\frac{dx}{dt} = x^2 + x \tag{3}$$

Here is the code implementation to solve this DE (plotting done in .m file, this is just the logic):

```
t_0 = 0; t_fin = 0.3;
n = 1001;
t_array = linspace(t_0, t_fin, n);
h = (t_fin - t_0)/n;

x = zeros(1,n);
x(1) = 1;

for i = 1:n-1
    x(i+1) = x(i) + (x(i)^2 + x(i))*h;
end
```

Graphical Output:

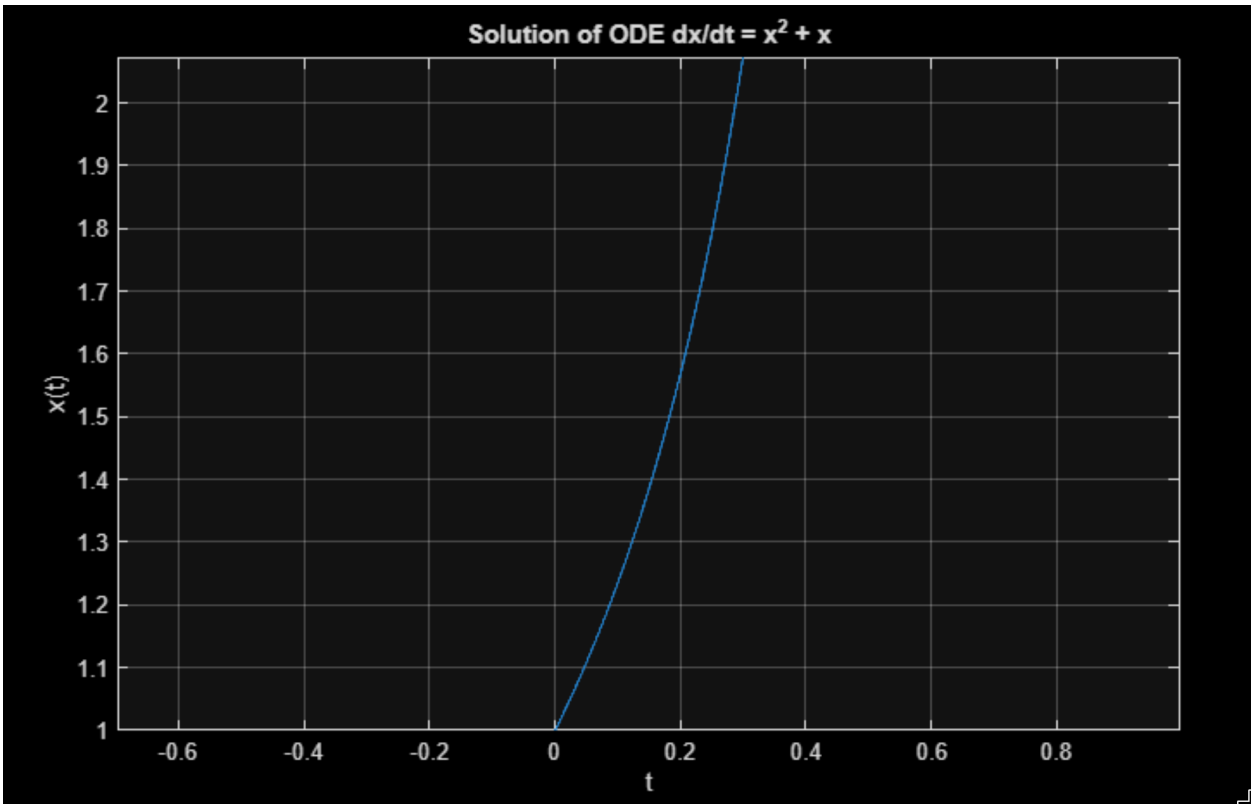


Figure 2: Solution of the Differential Equation

Interpretation:

Something interesting about this DE is that the system helps itself grow faster and faster. x represents the steady, linear progress while the x^2 term represents the compounding progress.

Q6) Higher Order Methods

a) Calculate the derivative of sin(x) using central difference. Make a log-log plot of error vs h. On one plot show Euler's method and the central difference method.

b) Write a new Matlab function that solves arbitrary ODEs using the midpoint method. Test it with $\dot{x} = x$, $x(0) = 1$, evaluated at $t = 1$. On one log-log plot show the error as a function of h for both the Euler method and the midpoint method.

Solution

a)

According to the central difference formula:

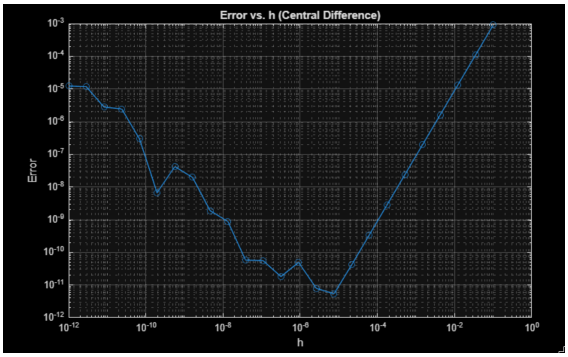
$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} \tag{4}$$

The derivative of sin(x) was calculated at x = 1.

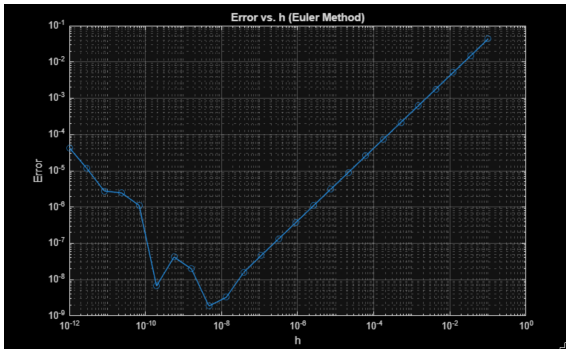
Code Block (For Calculating Derivative):

```
h = 1e-6;
derivative_at_x_1 = (sin(1 + h) - sin(1-h))/(2*h);
fprintf("By definition value of derivative of sinx at x = 1 is
(Central difference)");
disp(derivative_at_x_1);
```

On plotting central difference and Euler's method, the best h in the central difference case is in the 10^{-6} order while in Euler's case it is in the order of 10^{-8}. The error is much smaller in the case of central difference.



(a) Derivative using Central Difference



(b) Euler's Method

Figure 3: Comparison of Central Difference and Euler's Method

b)

Code Block (Logic for Mid-Point Method):

```
t_0 = 0; t_fin = 1;
n = 1001;
t_array = linspace(t_0, t_fin, n);
h = (t_fin - t_0)/n;
```

```
z = zeros(1,n);
z(1) = 1;
for i = 1:n-1
    k1 = z(i);
    z_mid = z(i) + (h/2)*k1;
    k2 = z_mid;
    z(i+1) = z(i) + h * k2;
end
```

Graphical Output:

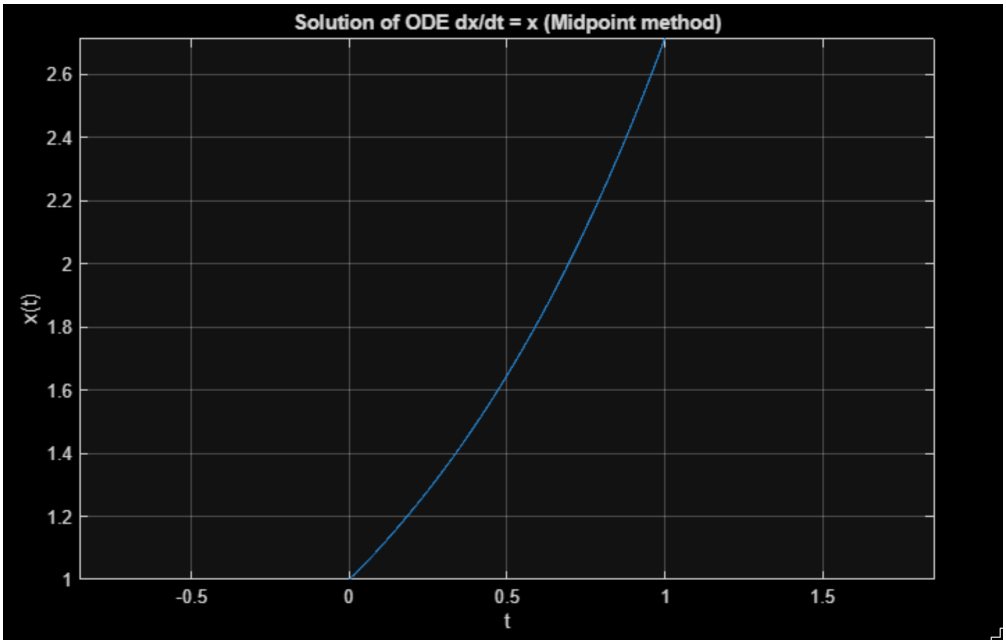


Figure 4: Solution of DE using Mid-Point Method



Figure 5: Error Comparison

Q7) Spring and Mass (2D)

One end of a negligible-mass spring (k, L_0) **is pinned to the origin, the other to a mass** (m)**. There is gravity** (g)**.** Initial Conditions: $\vec{r}_0 = x_0\hat{\imath} + y_0\hat{\jmath}$, $\vec{v}_0 = v_{x0}\hat{\imath} + v_{y0}\hat{\jmath}$.

a) Find the Equations of Motion (EoM).

b) Plot the trajectory and animate the trajectory of the mass.

c) Check the numerical solution in as many ways as possible (k=0, x0=0, g=0, etc.).

Solution

a)

The Equations of Motion are:

$$\begin{aligned} -k(\Delta l)\cos(\theta) &= ma_x \\ -k(\Delta l)\sin(\theta) - mg &= ma_y \end{aligned} \tag{5}$$

where

$$\Delta l = L - L_0 \tag{6}$$

$$\cos(\theta) = \frac{x}{\sqrt{x^2+y^2}}, \qquad \sin(\theta) = \frac{y}{\sqrt{x^2+y^2}} \tag{7}$$

$$a_x = \frac{d^2x}{dt^2}, \qquad a_y = \frac{d^2y}{dt^2} \tag{8}$$

The initial conditions are:

$$x(0) = 3, \quad y(0) = -4, \quad \dot{x}(0) = 5, \quad \dot{y}(0) = 9 \tag{9}$$

b)

To plot the path, I created a function named `mass_springRHS` with arguments z and p, where z is the state vector containing x, y, v_x, and v_y.

```
function zdot = mass_springRHS(~, z, p)
    m = p.m; g = p.g; k = p.k; L0 = p.L0;
    r_vec = z(1:2);
    v_vec = z(3:4);

    L = norm(r_vec);
    stretch = L - L0;
    T = k * stretch;
    er = r_vec/norm(r_vec);
    Tvec = -T * er;
    j_hat = [0; 1];
    a_vec = (1/m) * Tvec - g * j_hat;

    zdot = [v_vec; a_vec];
end
```

To solve it, I made use of the `ODEMidpoint` solver:

```
function [t_array, z_array] = ODEMidpoint(funct, timespan, z0, n)
    t0 = timespan(1); tfin = timespan(2);
```

```
    neqns = length(z0);
    h = (tfin - t0) / (n - 1);
    t_array = linspace(t0, tfin, n).';
    z_array = zeros(n, neqns);
    z_array(1, :) = z0(:).';
    for i = 1:n-1
        t = t_array(i);
        z = z_array(i, :).';
        k1 = funct(t, z);
        k2 = funct(t + h/2, z + (h/2) * k1);
        z_next = z + h * k2;
        z_array(i+1, :) = z_next.';
    end
end
```

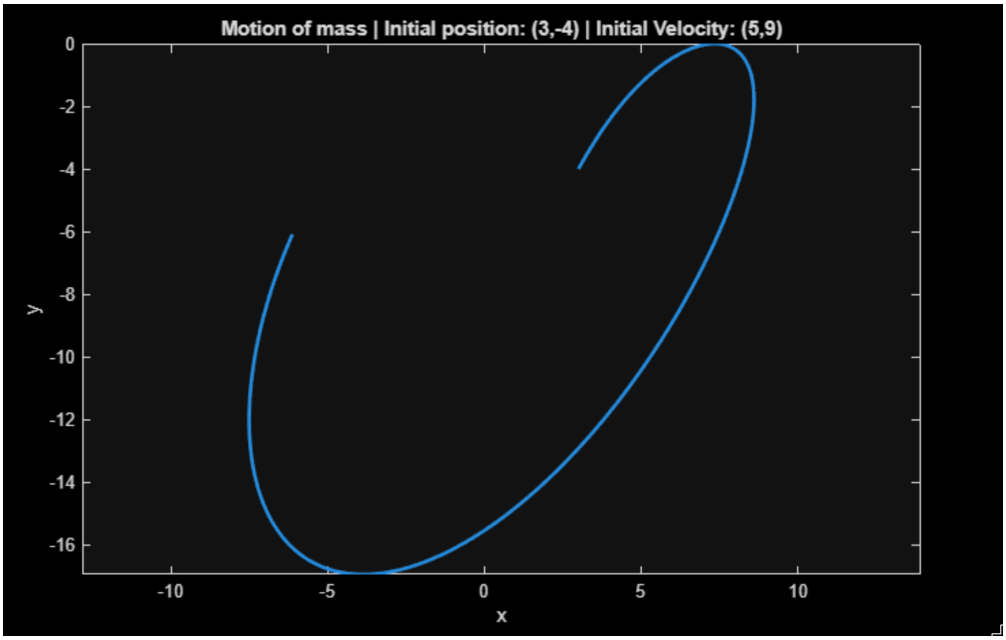


Figure 6: Plot of the Trajectory after the Animation

Q8) Cannon Ball

A cannon ball m is launched at angle θ and speed v_0. It is acted on by gravity g and a viscous drag with magnitude $|cv|$.

b) Find a numerical solution using $\theta = \pi/4$, $v_0 = 1$ m/s, $g = 1$ m/s^2, $m = 1$ kg, $c = 1$ kg/s. Compare the numeric and analytic solutions. At $t = 2$, how big is the error?

d) Use larger and larger values of v_0. As $v \to \infty$, what is the eventual shape?

e) For any given v_0, find the best launch angle θ^* for maximizing range. As $v_0 \to \infty$, to what angle does θ^* tend?

Solution

b)

For finding the numerical solution, I defined a function called `canon_ballRHS` with arguments z, m, g, c, where z is the state vector:

$$z = [x,\ y,\ v_x,\ v_y], \qquad \dot{z} = [\dot{x},\ \dot{y},\ \dot{v}_x,\ \dot{v}_y] \tag{10}$$

$$\dot{v}_x = -\frac{c}{m}v_x, \qquad \dot{v}_y = -g - \frac{c}{m}v_y \tag{11}$$

```
function zdot = canon_ballRHS(~,z,m,g,c)
    zdot = zeros(4,1);
    zdot(1) = z(3);
    zdot(2) = z(4);
    zdot(3) = -(c/m)*z(3);
    zdot(4) = -g-(c/m)*z(4);
end
```

c)

Comparing the numerical and analytical solutions:

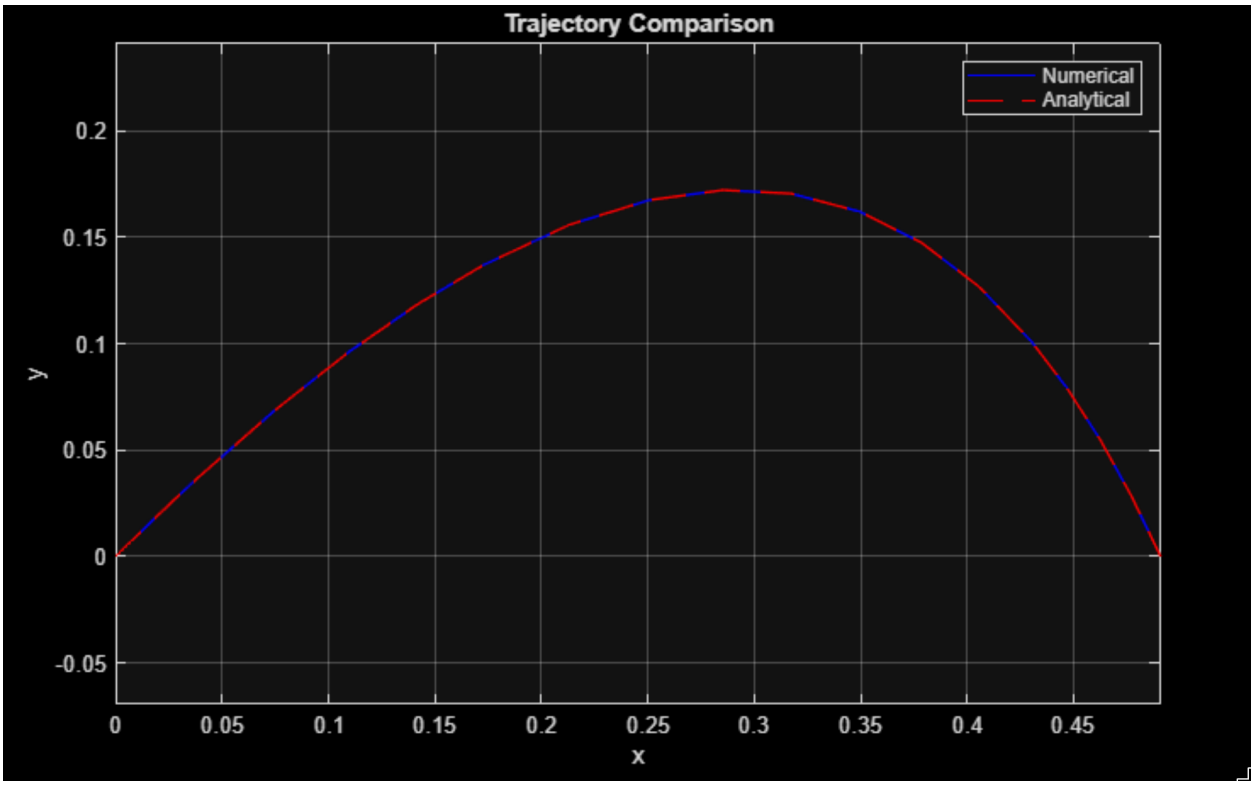


Figure 7: Comparison of Numerical and Analytical Solutions

The error at $t = 2$ comes out to be 4e-6.

d)

As the initial velocity was increased from 1 to 100, the path started to look considerably more linear:

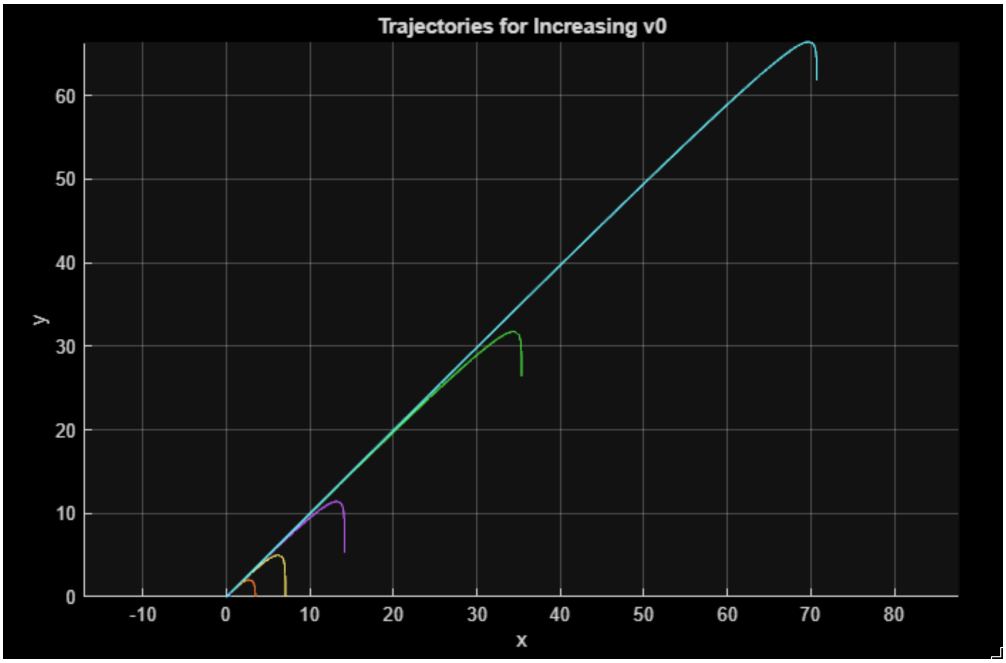


Figure 8: Trajectories on Increasing Initial Velocity

e)

To find the best launch angle that maximizes the range, the code tests 100 evenly spaced launch angles from 0° to 90°. For each angle, it simulates the full trajectory using `ode45` and finds where the ball first goes below ground ($y < 0$), recording the horizontal distance at that point.

```
v0 = 25;
theta = linspace(0, pi/2, 100);
theta_range = zeros(size(theta));

for i = 1:length(theta)
    v_x0 = v0 * cos(theta(i));
    v_y0 = v0 * sin(theta(i));
    z0 = [0; 0; v_x0; v_y0];

    [t, z] = ode45(@(t, z) canon_ballRHS(t, z, m, g, c), ...
                   timespan, z0, options);

    y = z(:,2);
    idx = find(y < 0, 1);

    if ~isempty(idx)
        theta_range(i) = z(idx,1);
    end
end
[~,ind] = max(theta_range);
theta_star = theta(ind);
disp(['Best angle = ', num2str(theta_star), ' radians']);
```

The best angle came out to be 0.15867 radians.

Q9) Statics vs Dynamics

2D equilibrium and dynamics of a point mass m at $\vec{r}_C$. Gravity $-g\hat{\jmath}$ points down. Points A and B are anchored at $\vec{r}_A$ and $\vec{r}_B$. Springs (constants k_A, k_B, rest lengths L_A, L_B) and dashpots (c_A, c_B) connect the mass to A and B. Air drag c_d.

a) Basic Statics: find an equilibrium position.

b) Basic Dynamics: given parameters and an initial condition, find the motion.

c) All equilibria: how many are there? What is the most/least number possible?

Solution

a)

To solve the problem, I defined a function named `mass_2D` with z and p as arguments, where $z = [x, y, v_x, v_y]$.

Parameters used:

$$k_A = 50,\ c_A = 2,\ L_A = 3, \qquad k_B = 50,\ c_B = 2,\ L_B = 3, \qquad c_d = 0.5 \tag{12}$$

I defined a function called `equilibriumResidual` with arguments r and p, where $x = r(1)$, $y = r(2)$, and $z = [x, y, 0, 0]$. Using `fsolve`, the equilibrium point obtained was:

$$x = 3.925, \quad y = 6.7004 \tag{13}$$

b)

Initial conditions chosen:

$$z = [5,\ 10,\ -9.895,\ 0] \tag{14}$$

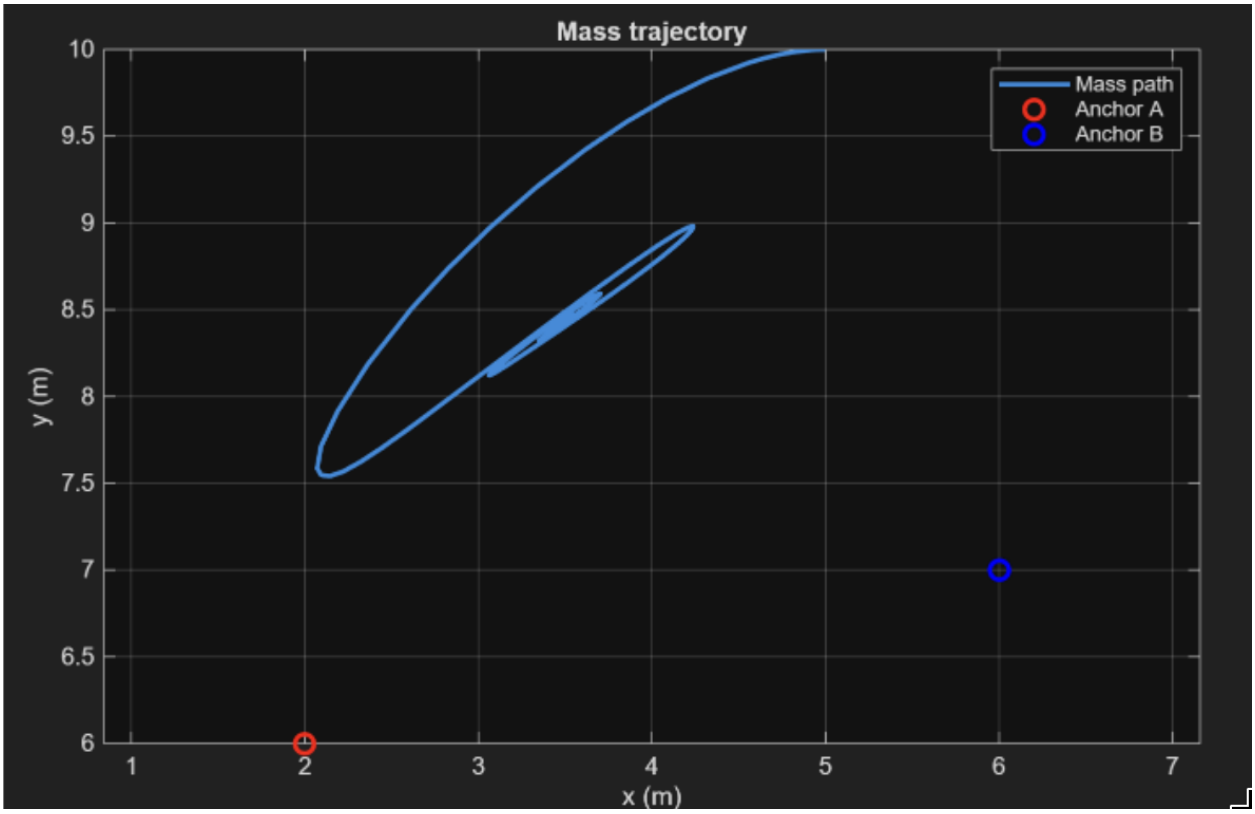


Figure 9: Trajectory

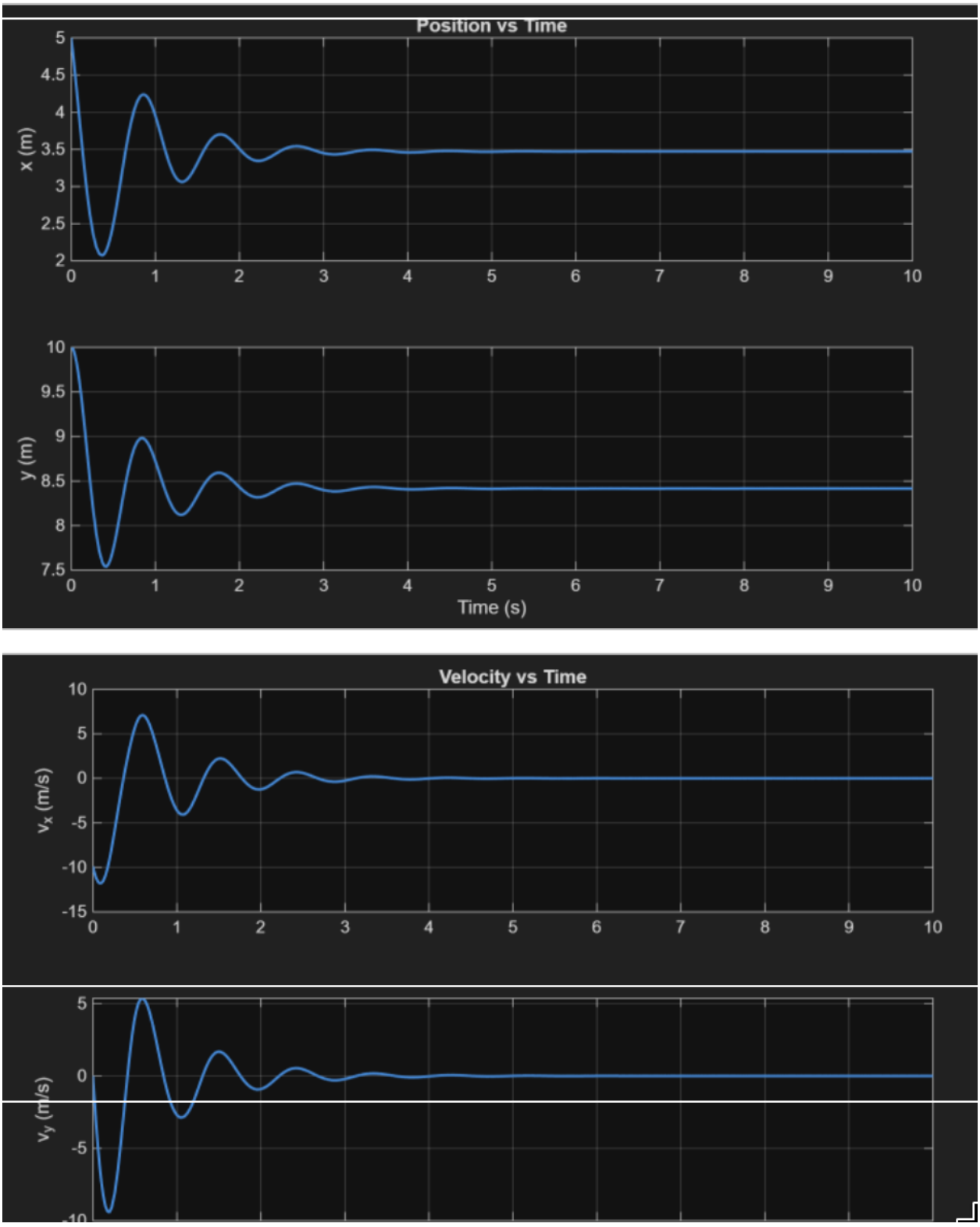


Figure 10: States vs Time Graphs

c)

To find multiple equilibria, I used the following set of initial guesses:

$$\text{guesses} = \{[0,0],\ [5,5],\ [10,5],\ [-5,5],\ [5,10],\ [8,2]\} \tag{15}$$

The code tries multiple different starting guesses and runs `fsolve` from each one. All solutions are collected, duplicates are removed, and what remains is the complete set of distinct equilibria.

```
opts = optimoptions('fsolve','Display','none');
eq_points = [];

for k = 1:size(guesses,2)
    try
        r_try = fsolve(@(r) equilibriumResidual(r,p), ...
                       guesses(:,k), opts);
        eq_points = [eq_points r_try];
    catch
    end
end
eq_points = unique(round(eq_points',4),'rows')';
```

Three equilibrium points were found:

$$[1.0497,\ 5.8764], \quad [3.4715,\ 8.4158], \quad [4.5197,\ 4.2068] \tag{16}$$

Q10) Linear ODEs

Find the general solution to the following ODEs by guessing:

b) $\dot{x} = -3x$, solve IVP $x(0) = 5$, compare with `ode45`.

c) $\ddot{x} = 9x$ — find the general solution.

d) $\ddot{x} = -4x$ — find the general solution.

Solution

b)

Taking the guess $x = Ce^{\lambda t}$ and substituting into the ODE with the initial condition $x(0) = 5$, we obtain $C = 5$ and $\lambda = -3$, giving the analytical solution $x(t) = 5e^{-3t}$.

The numerical solution was found using the `ode45` solver:

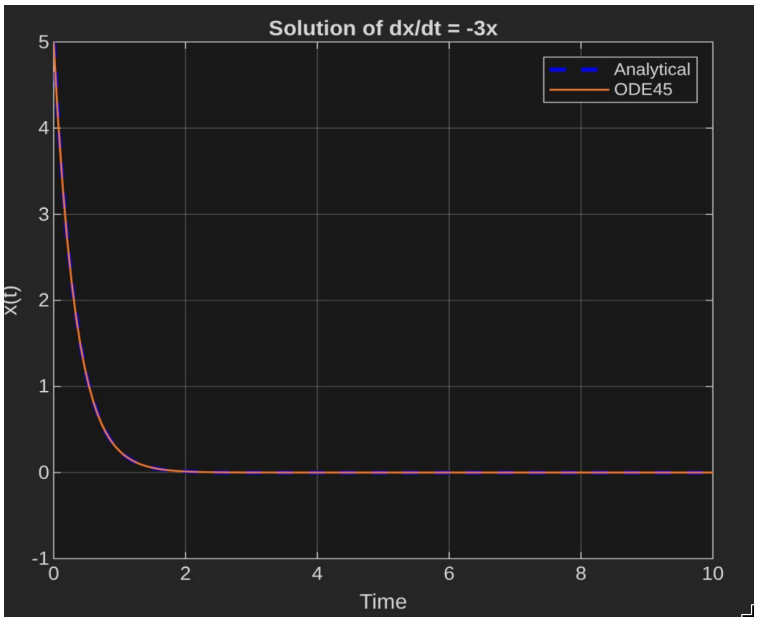


Figure 11: Comparison of Analytical and Numerical Solution

From the graph, a very accurate result is obtained. The deviation is only visible when zoomed in:

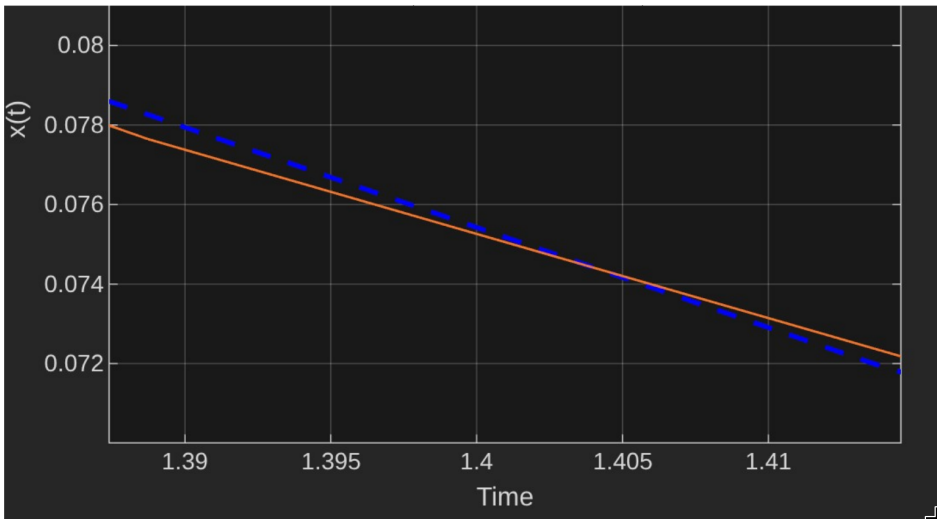


Figure 12: Zoomed In

c)

For this question, I took a linear combination of exponential functions as my guess:

$$x = c_1 e^{3t} + c_2 e^{-3t}, \qquad c_1 = c_2 = 0.5 \tag{17}$$

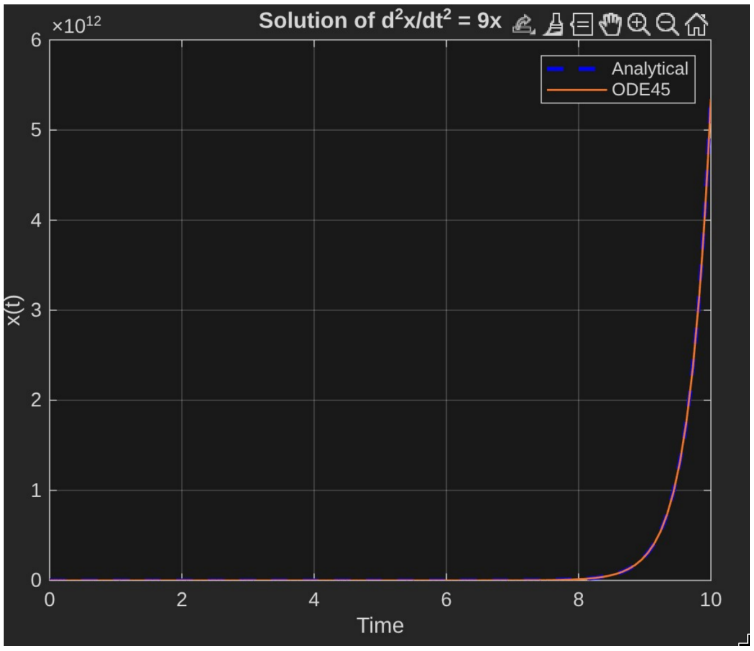


Figure 13: Comparison of Analytical and Numerical Solution

The analytical solution and `ode45` solution are in close agreement with little deviation:

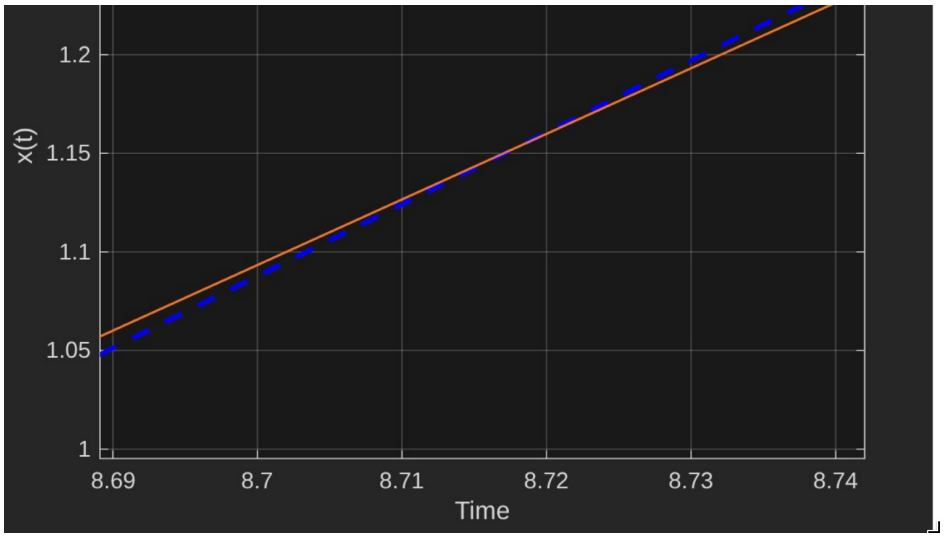


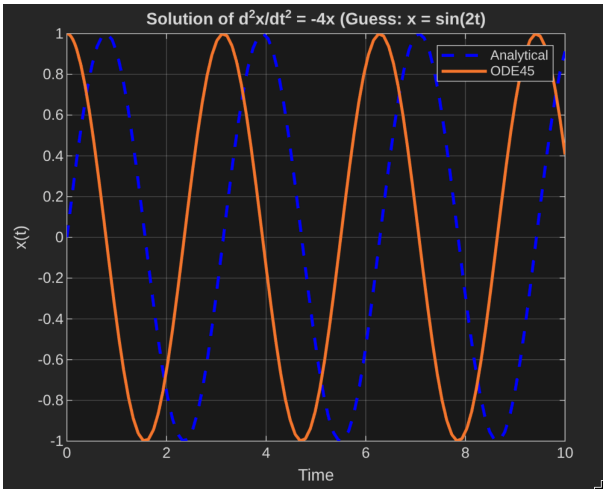
Figure 14: Zoomed In

d)

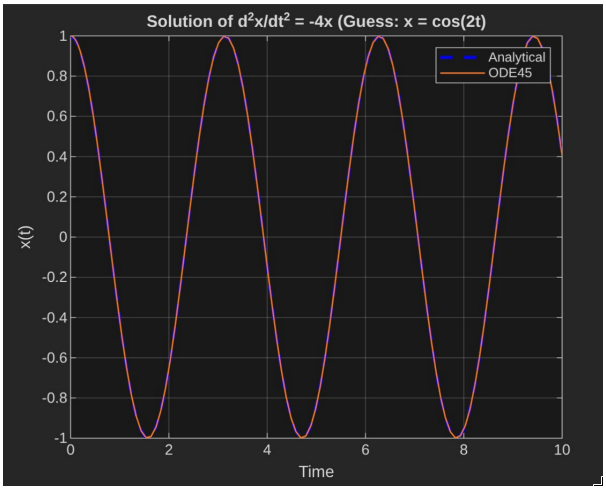
For this equation, I tried two guesses — a sine function and a cosine function — since this resembles the SHM equation:

$$x = \sin(2t), \qquad x = \cos(2t) \tag{18}$$

Graphical Outputs:



(a) When $x = \sin(2t)$



(b) When $x = \cos(2t)$

Figure 15: Comparison of Different Input Functions

Q11) Linear Stability of Springs and Mass

Back to the system of Q9.

a–c) Pick parameters and animate the motions, marking anchor points, spring rest-length circles, fixed points, and the path of the mass for different systems.

e–f) Using the Jacobian at a stable fixed point, solve the linearised equations and compare with `ode45`.

g) Find an unstable fixed point and repeat.

Solution

a–c)

Two systems with different parameters were considered.

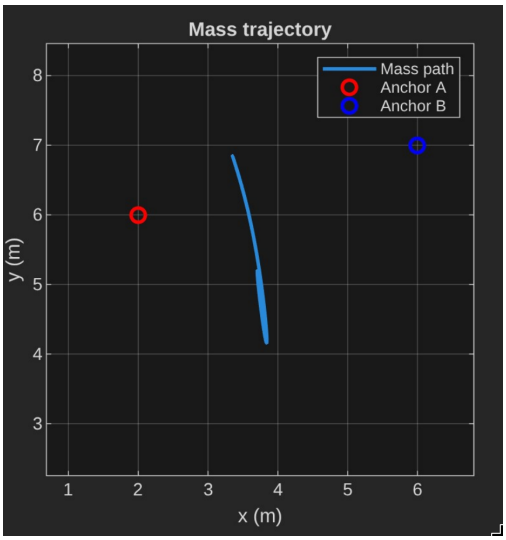
System 1 — Initial conditions:

$$z = [x_0,\ y_0,\ v_{x0},\ v_{z0}] = [3.35,\ 6.8,\ 0,\ 0] \tag{19}$$

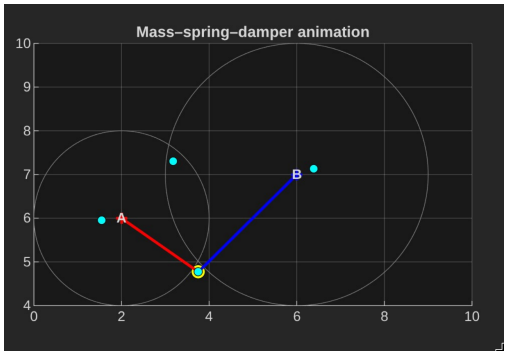
Anchor points: $A(2, 6)$, $B(6, 7)$. Spring rest lengths: $L_A = 2$, $L_B = 3$.

The mass converges to a stable equilibrium at $[3.7544,\ 4.7720]$. Using `fsolve`, four equilibrium points were found:

$$[1.5457,\ 5.9511],\quad [3.1788,\ 7.3059],\quad [3.7544,\ 4.7720],\quad [6.3895,\ 7.1316] \tag{20}$$



(a) Mass Trajectory — System 1



(b) Animation Frame

Figure 16: System 1 results and animation frame

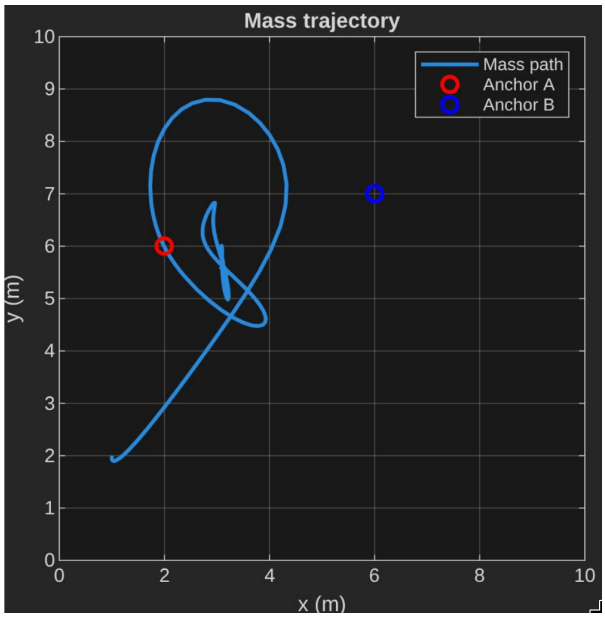
System 2 — Initial conditions:

$$z = [x_0,\ y_0,\ v_{x0},\ v_{z0}] = [1,\ 2,\ -1,\ -8] \tag{21}$$

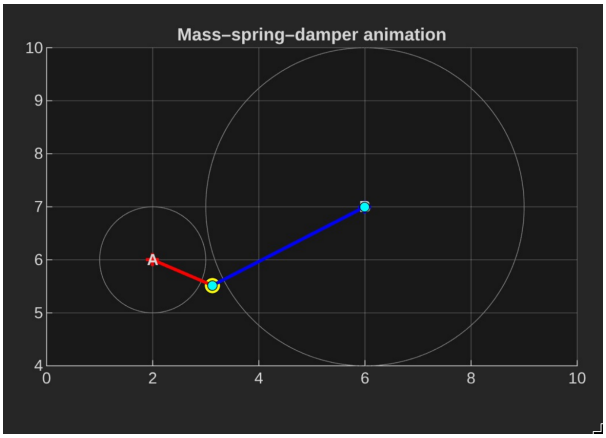
Anchor points: $A(2, 6)$, $B(6, 7)$. Spring rest lengths: $L_A = 1$, $L_B = 3$.

The mass converges to a stable equilibrium at $[3.7544,\ 4.7720]$. Using `fsolve`, two equilibrium points were found:

$$[3.1266,\ 5.5168], \quad [6,\ 7] \tag{22}$$



(a) Mass Trajectory — System 2



(b) Animation Frame

Figure 17: System 2 results and animation frame

d)

When a mass is close to an anchor point, the spring is almost infinitely compressed and can push outward with a large force in any direction. This enables the spring to balance gravity and the pull from the other spring.

e–f)

To solve for the linearised equations of motion near a fixed point, I defined a function `numericalJacobian`:

```
function A = numericalJacobian(f, z0)
    n = length(z0);
    A = zeros(n,n);
    eps = 1e-6;
    for i = 1:n
        dz = zeros(n,1);
        dz(i) = eps;
        A(:,i) = (f(z0 + dz) - f(z0 - dz)) / (2*eps);
    end
end
```

The Jacobian was computed at the equilibrium point and used to propagate the linearised solution:

```
A = numericalJacobian(@(z) mass2D(0,z,p), z_eq);
dz0 = [0.1; 0.1; 0; 0];
z0_near = z_eq + dz0;
t_lin = linspace(0,30,500);
z_lin = zeros(4,length(t_lin));
for i = 1:length(t_lin)
    z_lin(:,i) = expm(A*t_lin(i)) * dz0 + z_eq;
end
```

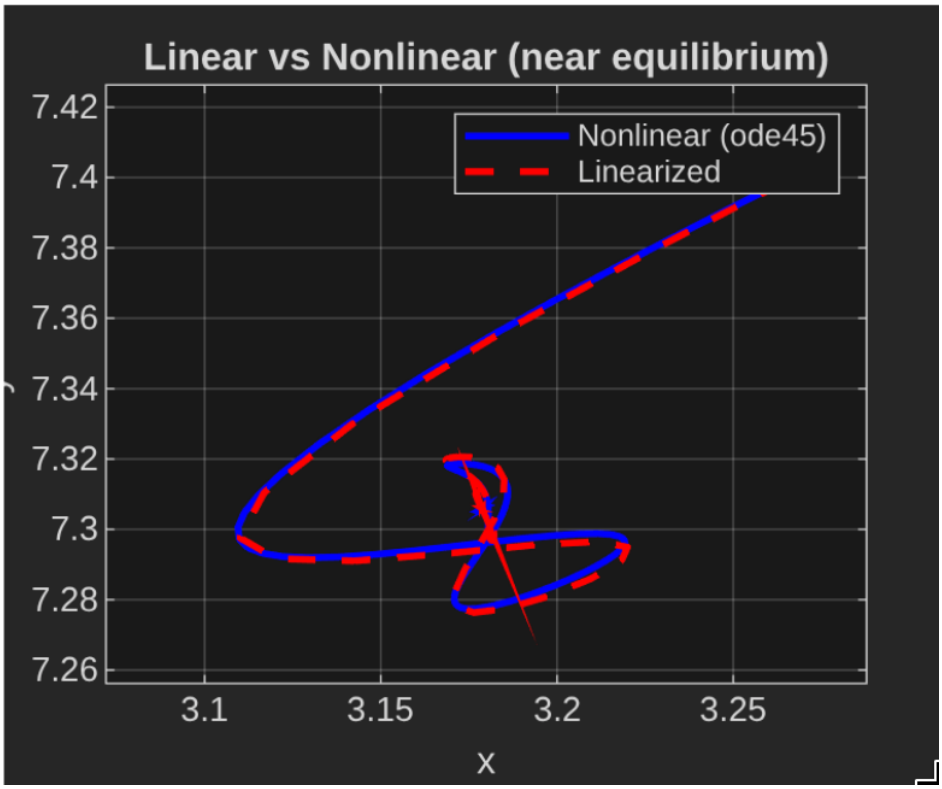


Figure 18: Nonlinear vs Linearized Solution

g)

To find an unstable equilibrium point, the Jacobian A is computed at each equilibrium and its eigenvalues are checked. If any eigenvalue has a positive real part, the equilibrium is unstable.

The point $[1.5457,\ 5.9511]$ was found to be unstable.

```
for i = 1:size(eq_points,2)
    z_eq_test = [eq_points(:,i); 0; 0];
    A_test = numericalJacobian(@(z) mass2D(0,z,p), z_eq_test);
    eigvals = eig(A_test);
    if any(real(eigvals) > 0)
        disp('Unstable equilibrium found:');
        disp(eq_points(:,i));
        z_eq_unstable = z_eq_test;
        break;
    end
end
```

Q12) Numerical Optimization

Consider the surface $z = (x(x-y))^2 - \exp(-x^2 - y^2) - \cos(x)$ **and the plane** $x + y + z/3 = 0$**.** Find the minimum value of z on the surface, on the plane, and inside the unit circle. Find the argmin (x, y). Do this with a grid search, then with `fmincon`. Make a 3D plot.

Solution

In the grid search, 301 points between -1 and 1 were taken for both x and y. The algorithm iterates through all combinations, evaluates the unit circle constraint, and updates the minimum value until convergence.

Grid search result:

$$x = 0.2467, \quad y = 0.9667, \quad z = -3.6437 \tag{23}$$

`fmincon` starts from an initial guess and iteratively moves downhill in the objective function while respecting the constraints.

`fmincon` result:

$$x = 0.2358, \quad y = 0.3642, \quad z = -1.799 \tag{24}$$

`fmincon` gives a far more accurate solution than grid search. Grid search is purely brute force and its precision depends entirely on the step size. `fmincon` uses gradients and actively moves toward the minimum.

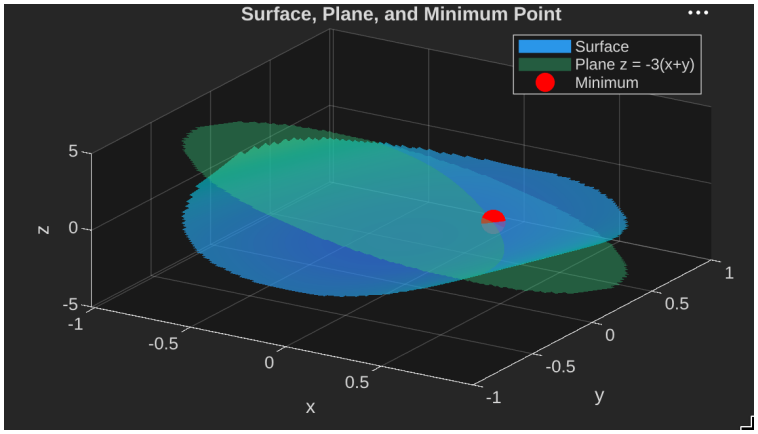


Figure 19: 3D Plot: Surface, Constraint Plane, and Minimum Point

Q13) Periodic Solutions

Finding periodic motions using root-finding.

a) Verify that $\ddot{\vec{r}} = -(k/m)\vec{r}$ gives periodic motion for all initial conditions.

b) Verify that $\ddot{\vec{r}} = -G(M/r^3)\vec{r}$ gives either escape or periodic (circle/ellipse/line) motion.

c–d) Consider $\ddot{\vec{r}} = -kr^{n-1}\vec{r}$ for $n \neq 1, -2$. Show straight-line motion from rest, circular motion with correct v_y, and non-periodic motion for random initial conditions.

e) Find a periodic motion that is not a line or circle using root finding.

Solution

a)

The following initial conditions were tested:

$$\text{ICs} = \{[0,1,2,3],\ [-1,3,7,0],\ [-2,10,-2,1],\ [0,0,1,1],\ [1,1,0,0]\} \tag{25}$$

For all of these initial conditions, only periodic motions were obtained:

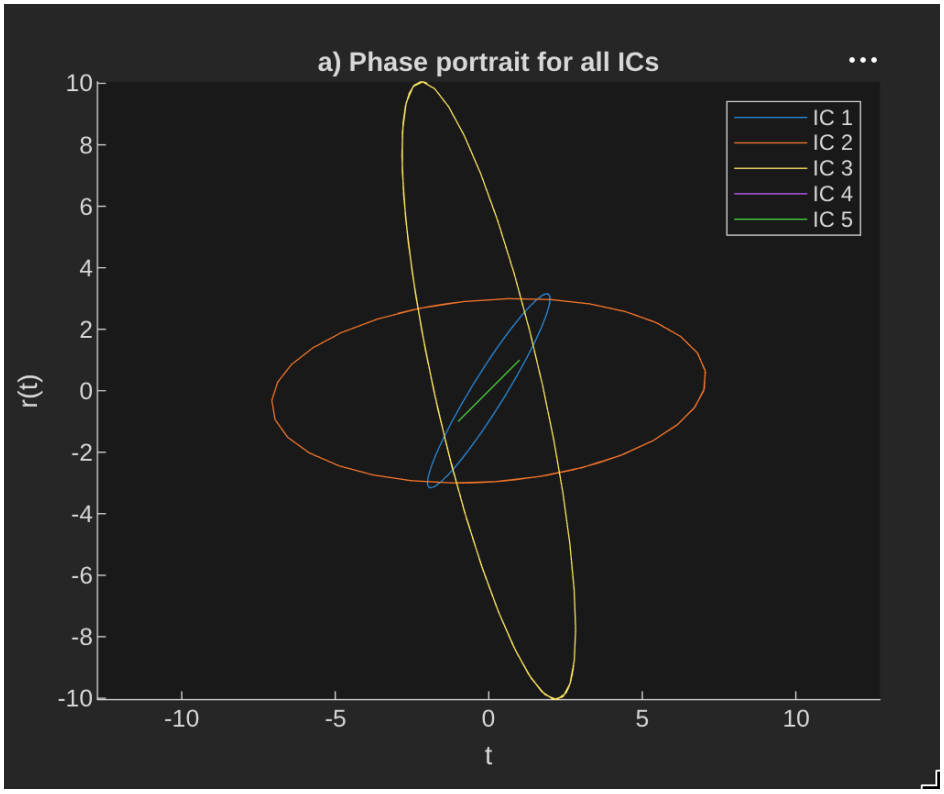


Figure 20: Motions at Different Initial Conditions

c)

The rest condition used is $[0, 0, 0, 1]$ and the other is $[1, 0, 0, 1]$, with $n = 3$:

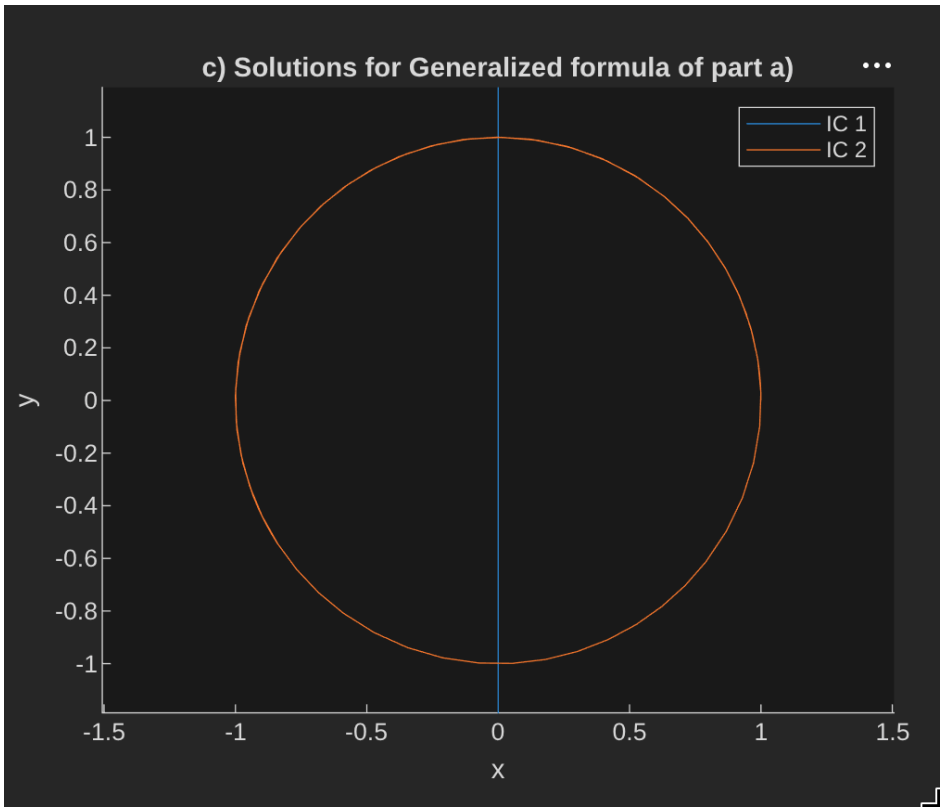
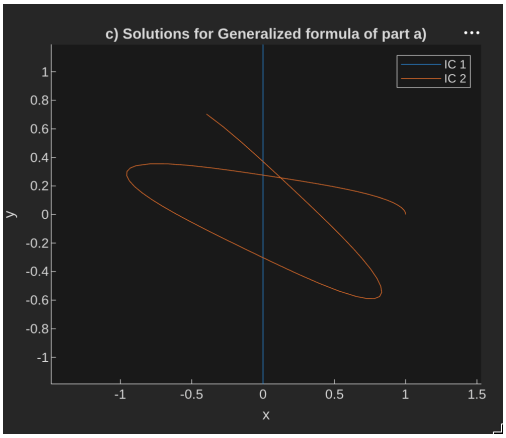
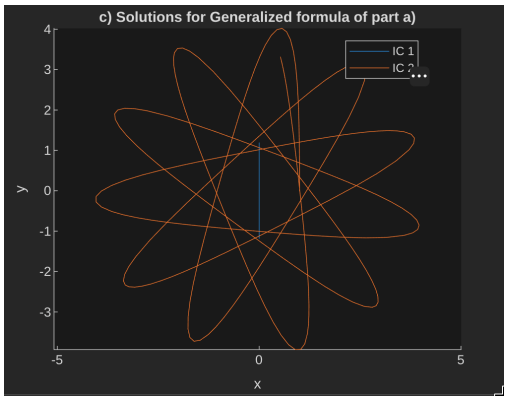


Figure 21: Straight-line motion from rest and circular motion with the other condition

At $v_y = 1$, circular motion was obtained. For a smaller value, an arbitrary shape resulted, and for a larger value, an aperiodic pattern emerged:



(a) At $v_y = 0.2$



(b) At $v_y = 12$

Figure 22: System behaviour at different velocity thresholds

d)

Keeping $n = 3$ but using random initial conditions:

$$\text{ICs} = \{[1, -2, 0, 4],\ [1, 3, 5, -2],\ [-4, 7, 1, -1],\ [-5, -3, -1, 0],\ [0, -2, -3, -5]\} \tag{26}$$

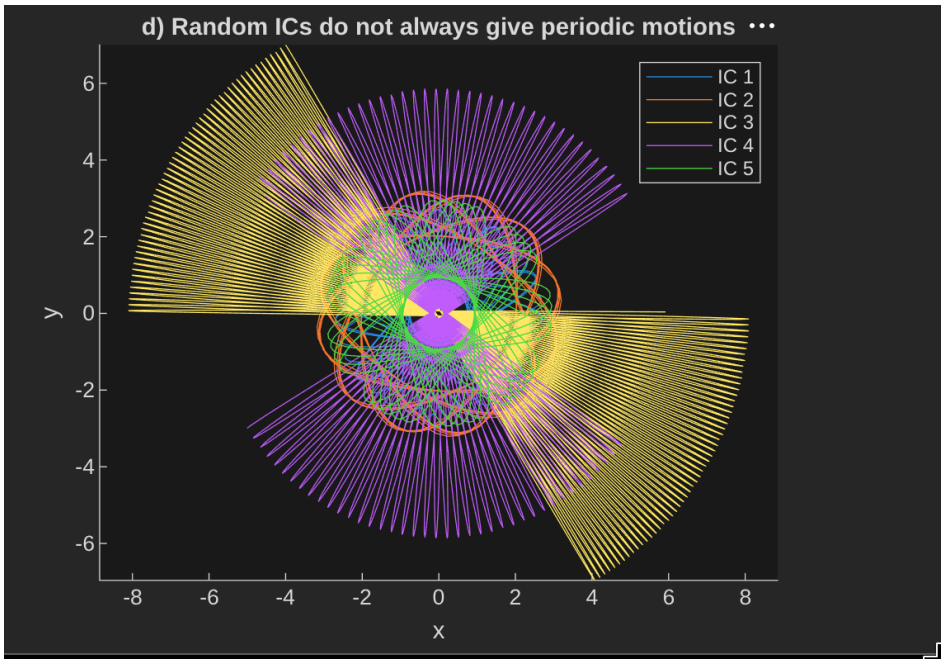


Figure 23: Non-Periodic Motions

Some of these motions represent quasi-periodic motion. For example, a planet that orbits every 10 seconds and wobbles every 7 seconds — since 10 and 7 share no common factor, the path never exactly repeats, and the surface defined by the motion becomes increasingly dense over time.

Question 15 — Initial Value Problem (IVP) with Collocation

Objective: This is the first of several problems on the general topic of trajectory optimization. Topics covered:

- Initial Value Problems (IVP), review
- Boundary Value Problems (BVP) via shooting
- Collocation Methods

Euler method solution of ODE (review). Consider:

$$\ddot{x} = -1, \quad t \in [0, 1] \tag{27}$$

a) Convert this to a first-order system:

$$\dot{x} = v, \quad \dot{v} = -1, \quad x(0) = 0, \quad v(0) = 0.5 \tag{28}$$

b) Implement forward Euler integration.

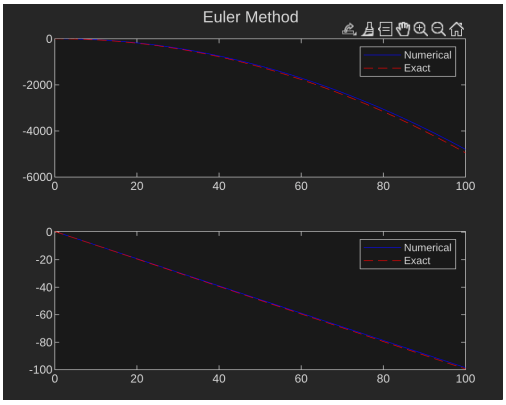
c) Implement Trapezoidal Integration.

d) Simulate the trajectory for different step-sizes (0.1 and 0.001 s) using both methods. What do you observe, and why?

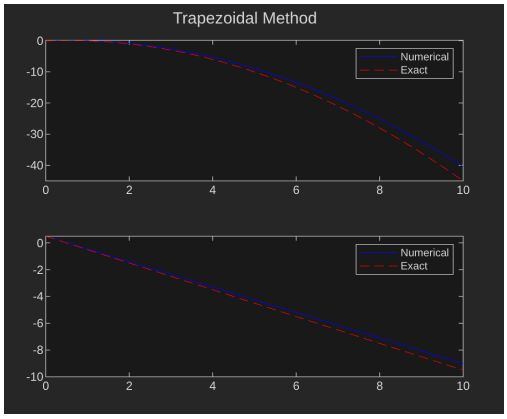
Solution

b–c)

For both Euler and Trapezoidal methods, I solved from $t_0 = 0$ to $t_{\text{fin}} = 100$ with step size $h = 0.1$.



(a) Output using Euler Method



(b) Output Using Trapezoidal Method

Figure 24: Comparison of Numerical Integration Methods

d)

Trajectories can be simulated by changing the parameters in the Euler and Trapezoidal functions. The error graph was used to visually compare the two methods.

```
h = 0.5;
[t1, x1, ~] = euler(0, 10, h);
```

```
[t2, x2, ~] = trapezoidal(0, 10, h);

x_exact1 = 0.5*t1 - 0.5*t1.^2;
x_exact2 = 0.5*t2 - 0.5*t2.^2;
```

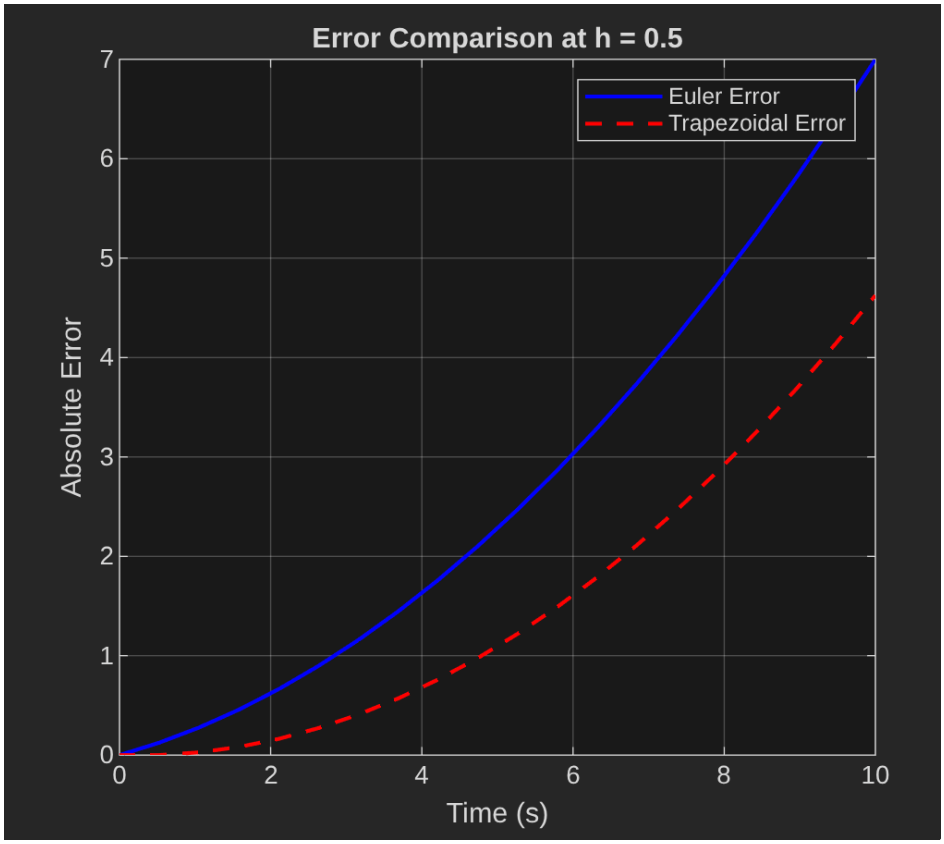


Figure 25: Error Comparison at $h = 0.5$

Question 16 — Boundary Value Problem via Shooting

$$\ddot{x} = -1, \quad x(0) = 0, \quad x(1) = 0 \tag{29}$$

Solve this with $h = 0.25$, then with $h = 0.1$.

a) Assume a value for s. Define the shooting function $F(s) = x(1;\, s)$.

b) Plot $F(s)$ vs s.

c) Use a home-made root-finding method (bisection/Newton) to find s^* such that $F(s^*) = 0$.

d) Verify your numerical result with an analytical solution.

Solution

a)

Defining the Shooting Function $F(s)$:

```
h = 0.1;
function F = shooting(s,h)
    t0 = 0; tfin = 1;
    N = round((tfin - t0)/h);
    t = linspace(t0, tfin, N);
    z = zeros(2,N);
    z(:,1) = [0,s];
    for i = 1:N-1
        z_dot = [z(2,i); -1];
        z(:,i+1) = z(:,i) + h*(z_dot);
    end
    F = z(1, end);
end
```

b)

On plotting the shooting function values vs s, a linear graph is obtained. This is because the analytical solution gives:

$$x(t) = -\frac{t^2}{2} + st \implies F(s) = x(1;\, s) = s - \frac{1}{2} \tag{30}$$

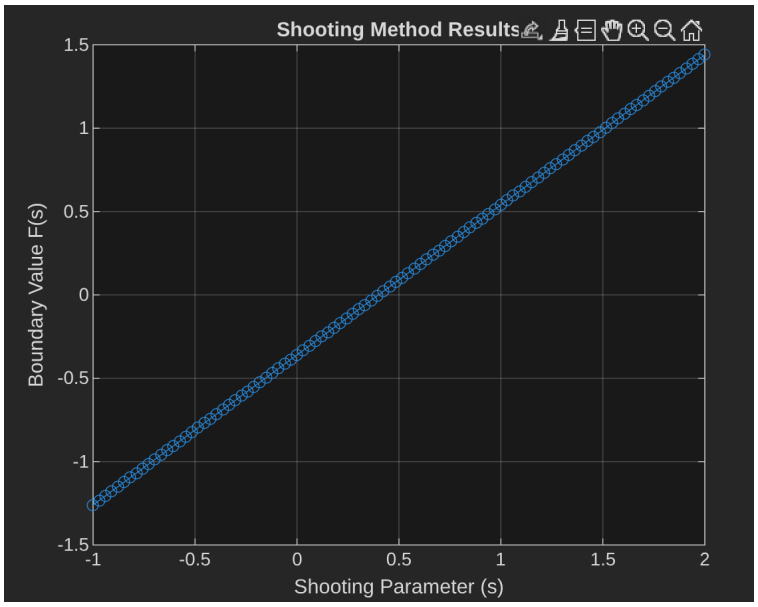


Figure 26: $F(s)$ vs s

c)

Newton-Raphson Method for root finding:

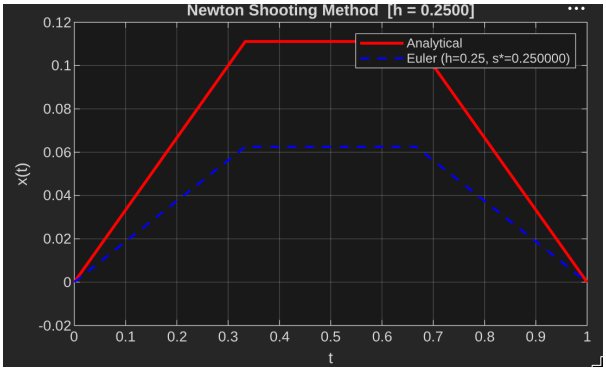
```
delta = 1e-5; s = 0;
tolerance = 1e-8; max_iter = 10000;

for i = 1:max_iter
    F1 = shooting(s,h);
    F_prime = (shooting(s+delta,h) - shooting(s-delta,h))/(2*delta);
    if abs(F1) < tolerance
        fprintf("Converged at s* = %.4f after %d iterations", s, i);
        break;
    end
    s = s - F1 / F_prime;
end
s_star = s;
```

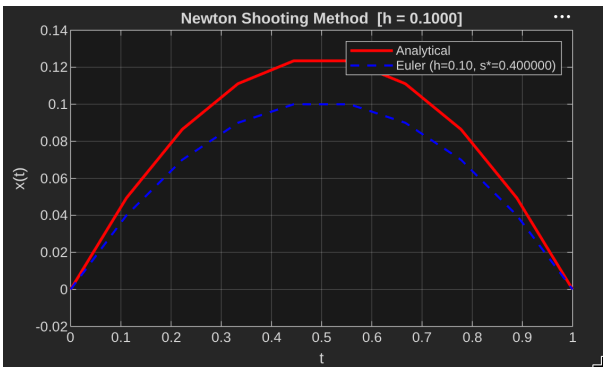
At higher step sizes the value of s is less accurate. As h decreases, s^* converges to the analytical value of 0.5:

$h = 0.25$	$s^* = 0.2$
$h = 0.1$	$s^* = 0.4$
$h = 0.01$	$s^* = 0.49$
$h = 0.0001$	$s^* = 0.4999$
Analytical	$s^* = 0.5$

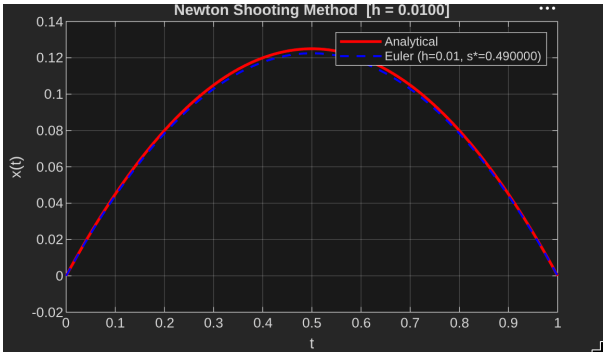
d)



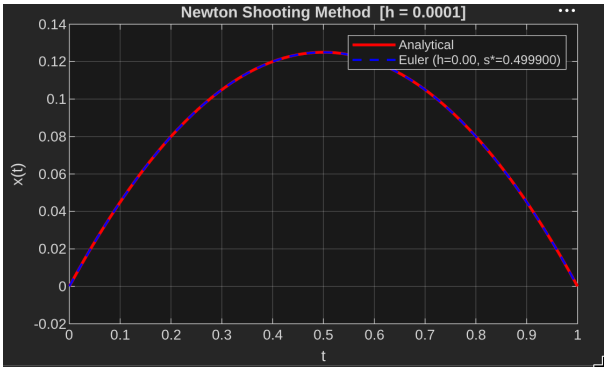
(a) $h = 0.25$



(b) $h = 0.1$



(c) $h = 0.01$



(d) $h = 0.0001$

Figure 27: Convergence Analysis Across Four Different Step Sizes

Question 17 — Collocation (Euler Discretization)

Consider the differential equations $\dot{x} = v$ and $\dot{v} = -1$, with boundary conditions $x(0) = 0$ and $x(1) = 0$. The Euler difference equations are:

$$x_{k+1} = x_k + hv_k, \qquad v_{k+1} = v_k - h \tag{31}$$

a) Formulate the linear system $Az = b$ where $z = [x_0, \ldots, x_N, v_0, \ldots, v_N]'$.

b) Enforce the two end conditions $x_0 = 0$, $x_N = 0$. For $N = 4$: A is 8×8, b is 8×1.

c) Solve the system using the backslash command ($z = A\backslash b$).

d) Compare with the shooting solution from the previous problem ($h = 0.25$).

e) Repeat with $N = 10$ using loops. Compare with the shooting solution ($h = 0.1$).

Solution

a–b)

```
N = 4; h = 1/N; n = N + 1; dim = 2*n;
A = zeros(dim); b = zeros(dim, 1); row = 1;

A(row, 1) = 1; b(row) = 0; row = row + 1;
A(row, N+1) = 1; b(row) = 0; row = row + 1;

for k = 0:N-1
    A(row, k+1) = -1; A(row, k+2) = 1;
    A(row, n + k+1) = -h; row = row + 1;
    A(row, n + k+1) = -1; A(row, n + k+2) = 1;
    b(row) = -h; row = row + 1;
end
```

c)

```
z = A\b;
x = z(1:n); v = z(n+1:end);
t = (0:N)' * h;
disp(table(t, x, v, 'VariableNames', {'t', 'x', 'v'}));
fprintf('\nRecovered v(0) = %.6f   (exact = 0.5)\n', v(1));
```

Graphical Output (with $N = 100$):

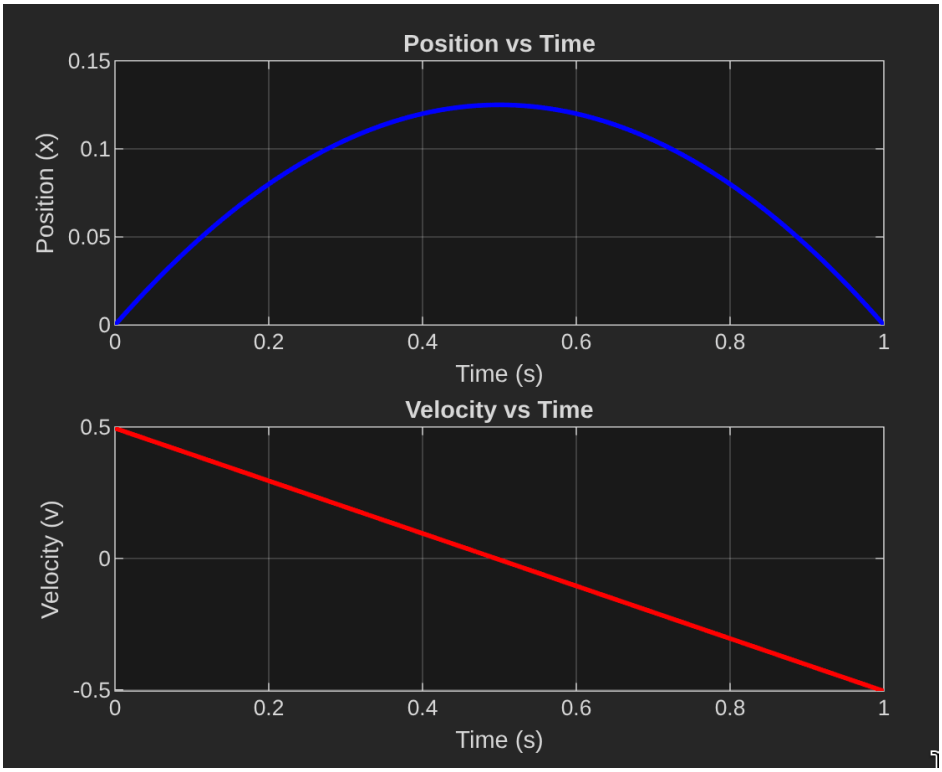


Figure 28: Position and Velocity plots with 100 collocation points. Higher N gives a smoother, more accurate solution.

Question 18 — Optimal Control

Move a mass $m = 1$ from rest at $x = 0$ to rest at $x = 1$ after $T = 1$ s, minimising energy:

$$\min_{u(t)} \int_0^1 u(t)^2 \, dt \tag{32}$$

subject to $\ddot{x} = u$, $x(0) = 0$, $v(0) = 0$, $x(1) = 1$, $v(1) = 0$.

a) Discretize and solve using `fmincon` (start with $N = 4$, then automate with loops).

b) Plot $x(t)$, $v(t)$, $u(t)$.

c) Admire the beautiful solution with large N.

Solution

a)

The code implementation is similar to the previous question. `fmincon` is used to solve the optimisation problem:

```
z0 = zeros(3*(N+1),1);
objective = @(z) sum(z(2*(n)+1).^2);
options = optimoptions('fmincon', 'Display', 'iter');
[z_opt, fval] = fmincon(objective, z0, [], [], A, b, ...
                        [], [], [], options);
x = z_opt(1 : N+1);
v = z_opt(N+2 : 2*(N+1));
u = z_opt(2*(N+1)+1 : end);
t = linspace(0, T, N+1);
```

b)

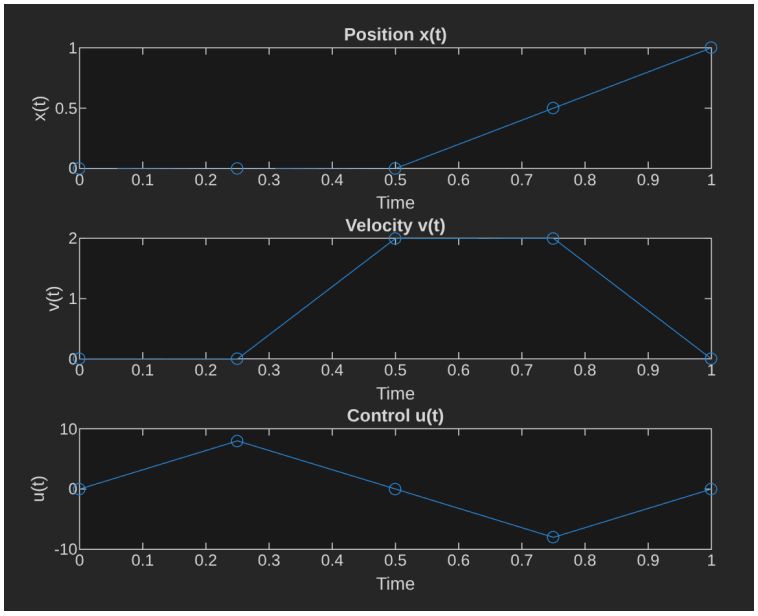


Figure 29: Position, Velocity and Control Input Graphs at $N = 4$

c)

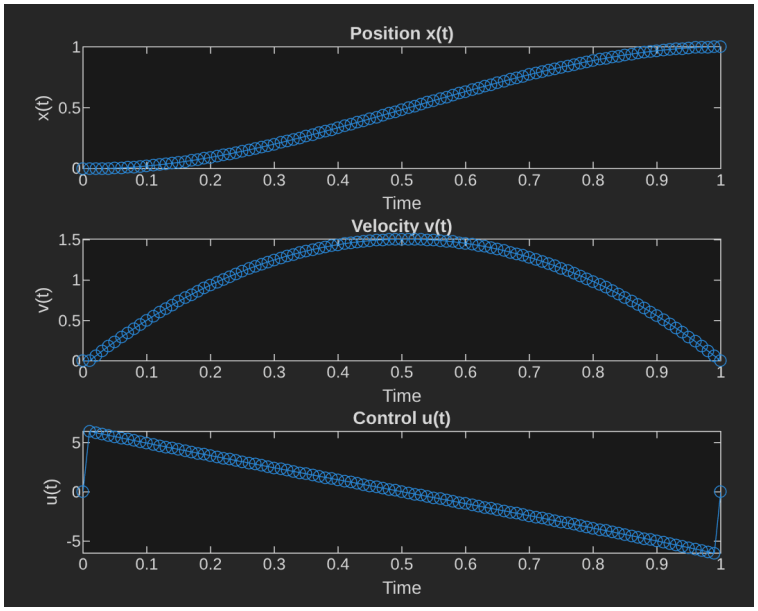


Figure 30: Position, Velocity and Control Input Graphs at $N = 100$

Question 19 — Trapezoidal Discretization

For the optimal control problem above, replace Euler discretization with trapezoidal discretization:

$$x_{k+1} = x_k + \frac{h}{2}(v_k + v_{k+1}) \tag{33}$$

a) Implement the scheme.

b) Compare the accuracy with the Euler solution. How can you estimate the accuracy?

Solution

a)

The key change from Euler to Trapezoidal in the matrix A:

```
for k = 0:N-1
    A(row, k+1) = -1; A(row, k+2) = 1;
    A(row, k+1 + (N+1)) = -h/2;
    A(row, k+2 + (N+1)) = -h/2;
    row = row+1;

    A(row, k+1 + (N+1)) = -1;
    A(row, k+2 + (N+1)) = 1;
    A(row, k+1 + 2*(N+1)) = -h/2;
    A(row, k+2 + 2*(N+1)) = -h/2;
    row = row + 1;
end
```

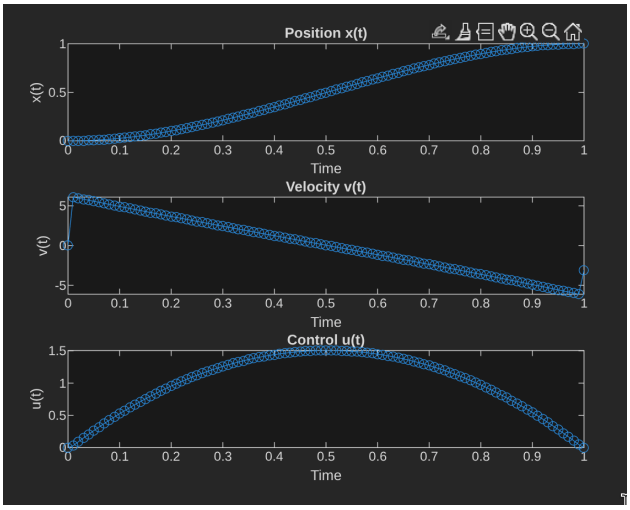


Figure 31: Position, Velocity and Control Input Graphs with Trapezoidal Discretization

Q20 — Mystery Matrix

Solution

Suppose we have a black-box function that applies a linear transformation $u = Av$, where A is an unknown 4×4 matrix:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \tag{34}$$

To determine the elements of A without analytical access, we evaluate the function using the standard basis vectors. Multiplying A by $e_1 = [1, 0, 0, 0]^T$ isolates the first column:

$$Ae_1 = \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ a_{41} \end{bmatrix} \tag{35}$$

Repeating for e_2, e_3, e_4 and concatenating the results reconstructs the full matrix — equivalent to passing the identity matrix through the function.

For non-linear systems, no global matrix A exists. Instead, we seek the local linear approximation: the Jacobian J. We construct it column-by-column using forward finite differences:

$$\Delta y = f(x_0 + \epsilon e_j) - f(x_0) \tag{36}$$

$$J_{*,j} \approx \frac{f(x_0 + \epsilon e_j) - f(x_0)}{\epsilon} \tag{37}$$

The resulting matrix J can be analysed via its eigenvalues to determine local stability.

Q21 — Optimal Planning and Control

Consider a 2D point-mass model (drone/lander):

$$\dot{x} = v_x, \quad \dot{z} = v_z, \quad \dot{v}_x = u_x, \quad \dot{v}_z = u_z - g \tag{38}$$

where $g = 1.62$ m/s^2.

Initial condition: $x(0) = x_0,\ z(0) = z_0,\ v_x(0) = v_{x0},\ v_z(0) = v_{z0}$

Final condition: $x(T) = 0,\ z(T) = 0,\ v_x(T) = 0,\ v_z(T) = 0$

Control constraint: $\sqrt{u_x^2 + u_z^2} \leq u_{\max}$

a) Solve using Euler or trapezoidal collocation with `fmincon`, with $T = 5$ s and initial condition $x_0 = 5$, $z_0 = 10$, $v_{x0} = -1$, $v_{z0} = -2$.

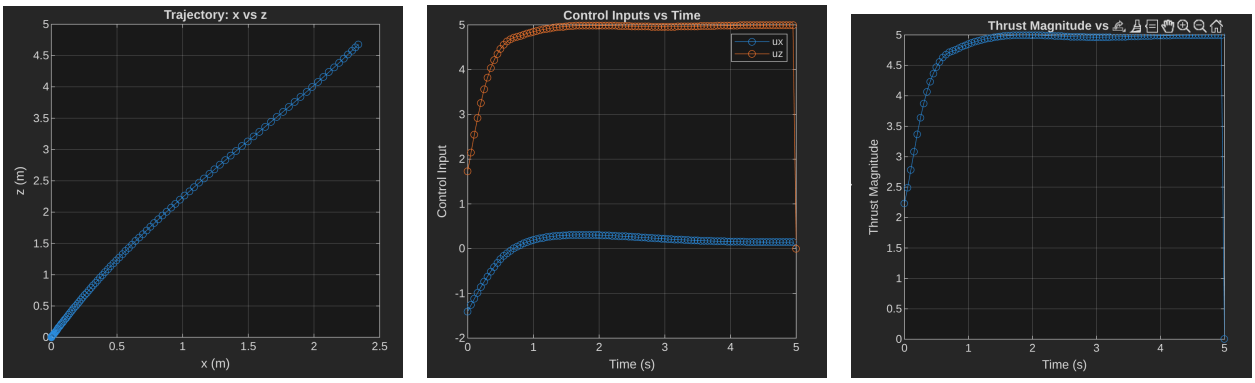
b) Plot trajectory, control inputs, and thrust magnitude vs time.

Solution

a)

A function called `Euler_Discretization` was written separately to avoid code repetition.

b)



(a) Trajectory Plot (b) Control Inputs (c) Thrust Magnitude

Figure 32: System Response and Control Metrics for Q21

The intuition behind the control inputs graph: in the vertical direction, the drone is moving downward at 2 m/s and gravity is also pulling it down, so u_z must be applied quickly at a high magnitude — hence it saturates. The drone moves horizontally at 1 m/s in the negative x-direction, which is already favourable for reaching $x = 0$ from $x = 5$, so little horizontal thrust is needed.

Q22 — Open-Loop Execution

a) Simulate the system forward using the computed control.

b) Introduce disturbance: $u_{\text{applied}} = \alpha u^*$, $\alpha \in [0.7, 0.9]$.

c) Plot planned vs actual trajectory and time vs states.

d) Compute final error.

Solution

a–b)

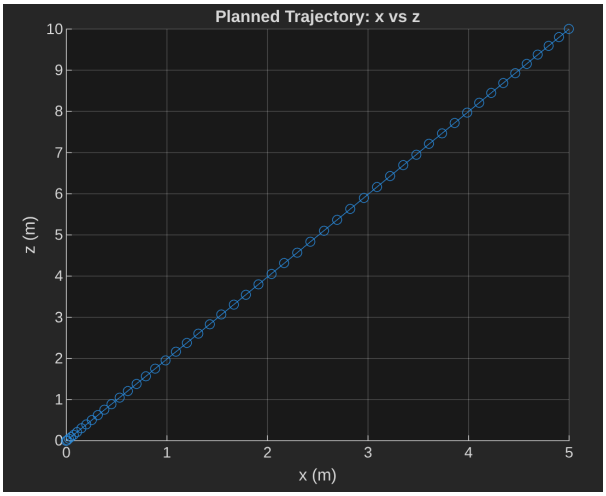
Forward control is simulated and the applied disturbance is incorporated:

```
alpha = 0.7 + (0.9 - 0.7) * rand(1,1);

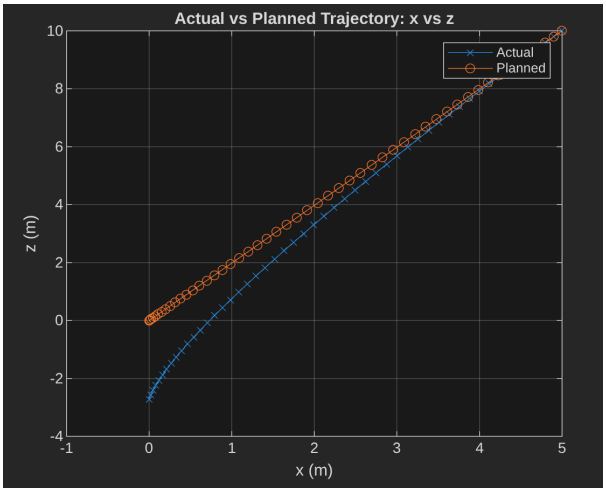
Ux_applied = alpha*Ux;
Uz_applied = alpha*Uz;

for k = 1:N
    x_actual(k+1) = x_actual(k) + h*Vx_actual(k);
    z_actual(k+1) = z_actual(k) + h*Vz_actual(k);
    Vx_actual(k+1) = Vx_actual(k) + h*Ux_applied(k);
    Vz_actual(k+1) = Vz_actual(k) + h*(Uz_applied(k) - g);
end
```

c)



(a) Trajectory Plot



(b) Planned vs Actual

Figure 33: Trajectory analysis under open-loop control. The right-hand plot shows that the spacecraft would crash, illustrating the limitations of open-loop control.

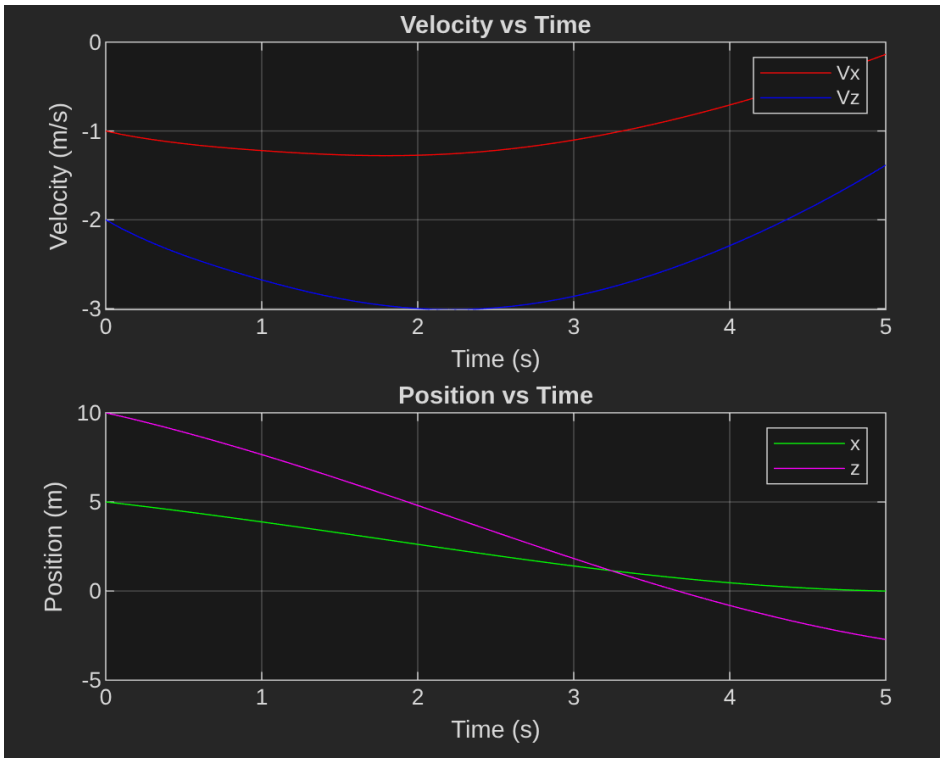


Figure 34: Time vs States

d)

The final error was computed using the root mean square error between the planned and actual trajectories:

```
error_x = x_actual - x;
error_z = z_actual - z;
rmse_x = sqrt(mean(error_x.^2));
rmse_z = sqrt(mean(error_z.^2));
fprintf('RMSE for x: %.4f m\n', rmse_x);
fprintf('RMSE for z: %.4f m\n', rmse_z);
```

The root mean square errors were:

$$\text{RMSE}_x = 0.0631 \text{ m}, \qquad \text{RMSE}_z = 1.1367 \text{ m} \tag{39}$$

Q23 — MPC Tracking (Feedback Control)

Use the trajectory from Q21 as reference. At each time step, solve:

$$\min \sum_{i=1}^{N_p} (x - x^{\text{ref}})^\top Q\,(x - x^{\text{ref}}) + u^\top R\,u \tag{40}$$

subject to dynamics and constraints, with step size 100 ms.

a) Implement MPC with horizon N_p.

b) At each step: solve optimisation, apply only the first control.

c) Use the same disturbance as Q22 ($\alpha \in [0.7, 0.9]$).

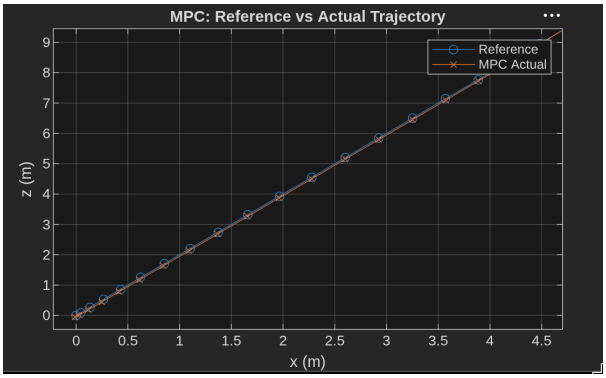
d) Plot reference vs MPC trajectory and control comparison.

e–h) Repeat for $N_p \in \{5, 10, 20, 30\}$ and compare tracking error.

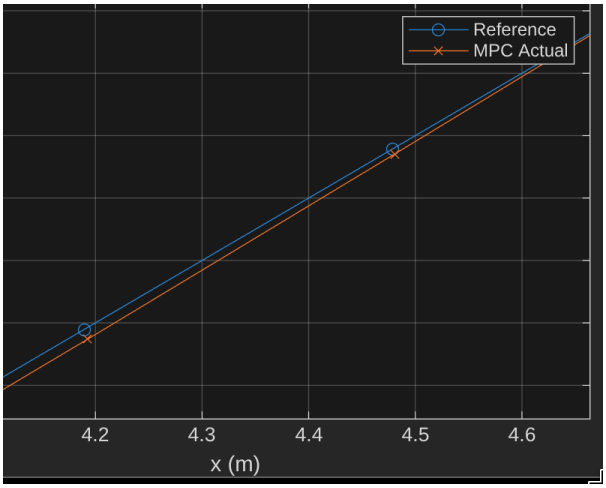
Solution

d)

On implementing MPC with the specified horizon and disturbance, the following results were obtained:



(a) Trajectory Comparison



(b) Zoomed-In View

Figure 35: Trajectory tracking performance. The zoomed-in view shows that the actual trajectory with feedback closely follows the reference.

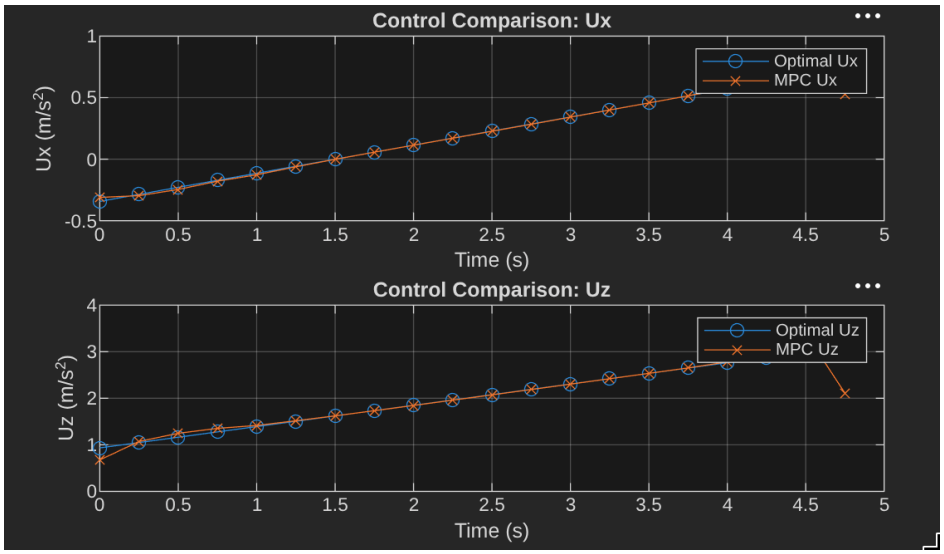


Figure 36: Comparison of Control Inputs

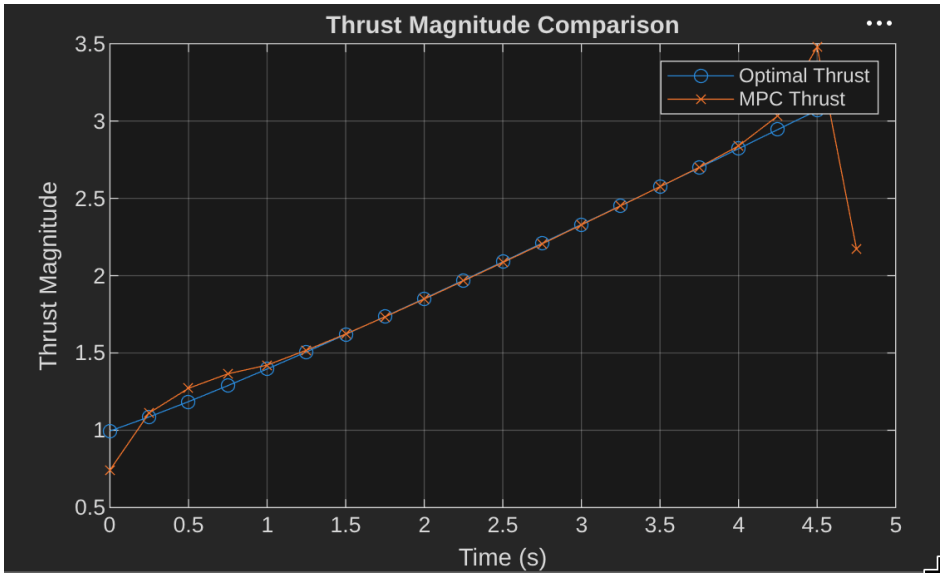


Figure 37: Thrust Magnitude

Q24 — Time-Optimal Control

Objective: minimise final time T:

$$\min_{T} \; T \tag{41}$$

a) Formulate the problem with T as a decision variable.

b–c) Use trapezoidal collocation and solve using nonlinear optimisation.

d) Plot trajectory, control inputs, and thrust magnitude.

e) Compare with the minimum-energy solution from Q21.

Solution

a–d)

With the objective being to minimise time, T is introduced as a decision variable. The function signature becomes:

$$[t, x, z, V_x, V_z, U_x, U_z, T_{\text{opt}}] = \texttt{TimeOptimal}(N,\ u_{\max}) \tag{42}$$

The state vector w is:

$$w = [x_0, \ldots, x_N,\ z_0, \ldots, z_N,\ v_{x0}, \ldots, v_{xN},\ v_{z0}, \ldots, v_{zN},\ u_{x0}, \ldots, u_{xN},\ u_{z0}, \ldots, u_{zN},\ T] \tag{43}$$

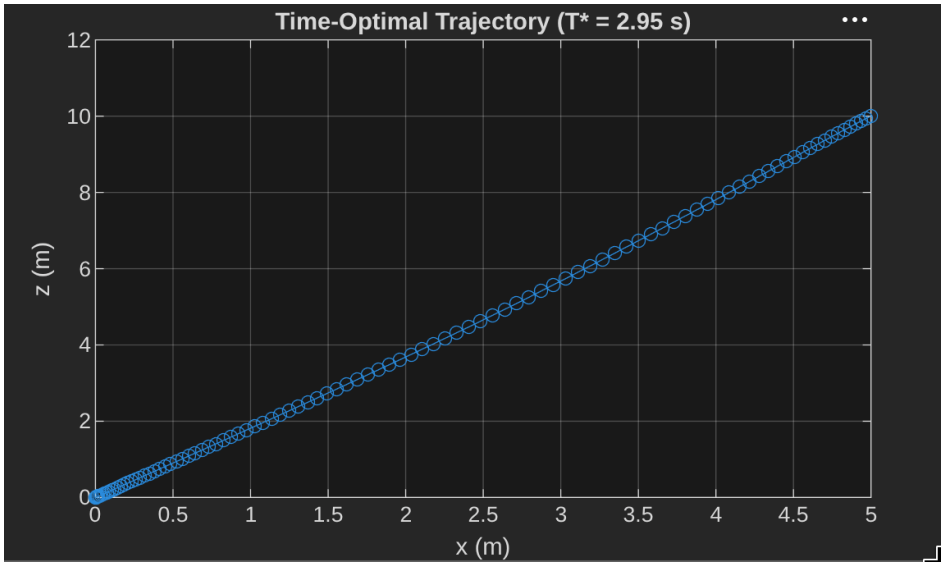


Figure 38: Optimal trajectory. The optimal time comes out to be 2.95 s with 100 collocation points.

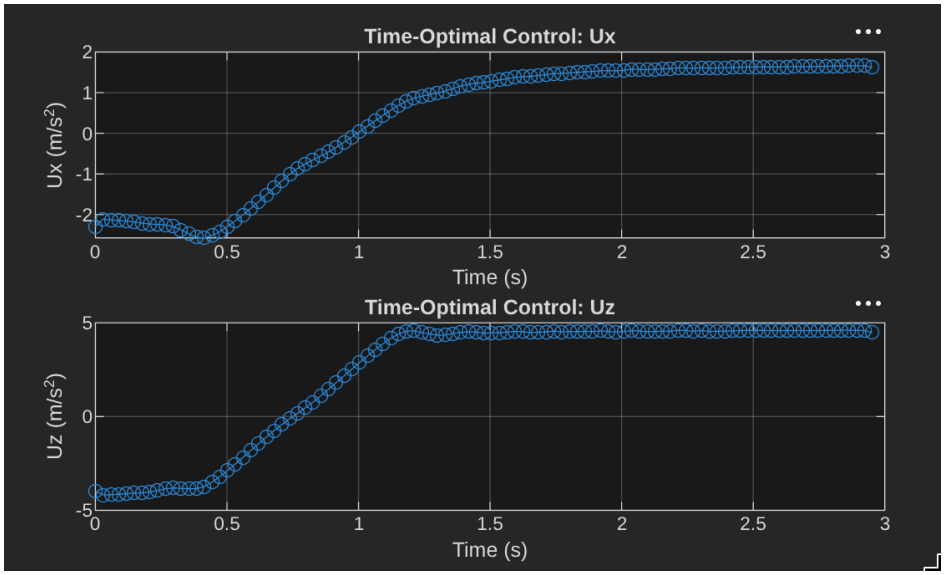


Figure 39: Control Inputs

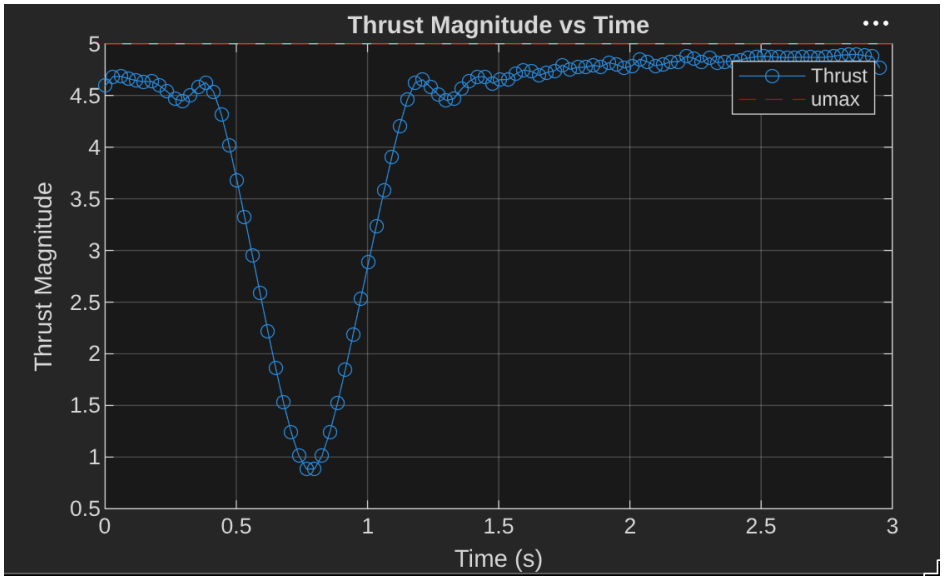


Figure 40: Thrust Magnitude

e)

Comparing the minimum-time and minimum-energy solutions:

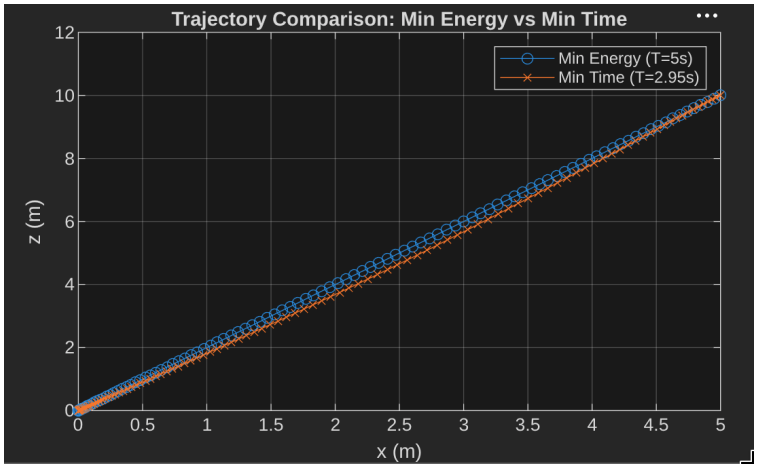


Figure 41: The paths differ slightly — the time-optimal path is more concave than the minimum-energy solution.

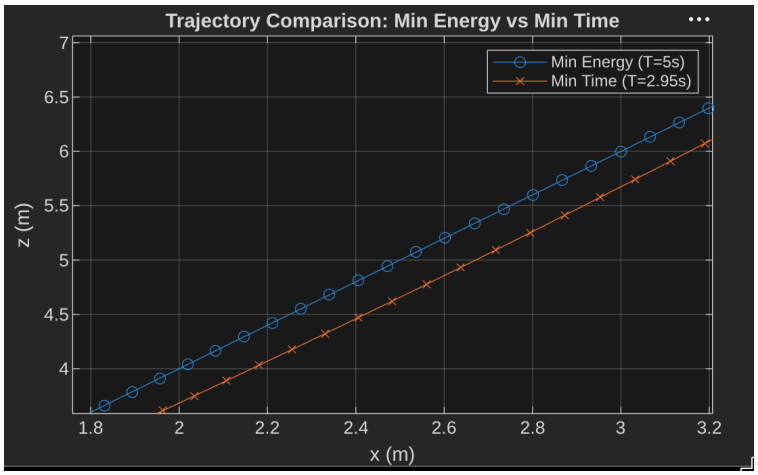


Figure 42: Zoomed In

u_z is more saturated than u_x, staying at 5 m/s^2 for most of the flight. The intuition is that to minimise time, the drone must use maximum available thrust — hence the control saturates throughout.